

Parte 2
INICIO

Le mieux est l'ennemi du bien (Lo mejor es enemigo de lo bueno.)

Voltaire

Objetivos

- Definir la etapa de inicio.
- Motivar los capítulos siguientes de esta sección.

Introducción

Este capítulo define la fase de inicio de un proyecto. Si las ideas del proceso no son del interés del lector, o prefiere centrarse en primer lugar en aprender la actividad práctica principal de esta fase —modelado de casos de uso— entonces puede saltarse este capítulo.

La mayoría de los proyectos requieren una etapa inicial breve en la que se estudian los siguientes tipos de preguntas:

- ¿Cuál es la visión y el análisis del negocio para este proyecto?
- ¿Es viable?
- ¿Comprar y/o construir?
- Estimación aproximada del coste: ¿cuesta 10K-100K o millones de euros?
- ¿Deberíamos abordarlo o no seguir?

Para definir la visión y obtener una estimación (de la que no te puedes fiar) del orden de magnitud es necesario llevar a cabo alguna exploración de los requisitos. Sin embargo, el objetivo de la etapa de inicio no es definir todos los requisitos, o generar una estimación creíble o plan de proyecto. Aún a riesgo de simplificar demasiado, la idea es hacer la investigación justa para formar una opinión racional y justificable del propósito global y la viabilidad del nuevo sistema potencial, y decidir si merece la pena invertir en un estudio más profundo (el objetivo de la fase de elaboración).

Por tanto, la fase de inicio debería ser relativamente corta en la mayoría de los proyectos, una duración de una a unas pocas semanas. De hecho, en muchos proyectos, si la duración es superior a una semana, entonces se pierde la idea fundamental de la etapa de inicio: decidir si merece la pena una investigación seria (durante la elaboración), no llevar a cabo esta investigación.

La fase de inicio en una frase:
Vislumbrar el alcance del producto, visión y análisis del negocio.

El principal problema resuelto en una frase:
¿Está de acuerdo el personal involucrado en la visión del proyecto, y merece la pena invertir en un estudio serio?

4.1. Inicio: una analogía

En el negocio del petróleo, cuando se está considerando una nueva zona, algunos de los pasos que hay que abordar son:

- 1. Decidir si hay evidencias suficientes o un análisis del negocio que justifique perforaciones de exploración.
- 2. De ser así, llevar a cabo medidas y perforaciones de exploración.
- 3. Proporcionar información del alcance y estimación.
- 4. Pasos adicionales...

La fase de inicio es como el primer paso en esta analogía. En el primer paso, no se predice cuánto petróleo hay, o el coste o esfuerzo para extraerlo. Es prematuro —no hay suficiente información—. Aunque estaría bien ser capaz de responder a las preguntas de “cuánto” y “cuándo” sin el coste de la exploración, en el negocio del petróleo se entiende que no es realista.

En términos del UP, el paso de exploración realista se corresponde con la fase de elaboración. La fase de inicio que la precede es parecida a un estudio de viabilidad para decidir si incluso merece la pena invertir en perforaciones de exploración. Sólo después de la exploración (elaboración) tenemos los datos y los conocimientos para hacer, de algún modo, planes y estimaciones creíbles. Por tanto, en el desarrollo iterativo y el UP, los planes y estimaciones de la fase de inicio no deben considerarse fiables. Simplemente proporcionan una percepción del orden de magnitud del grado de esfuerzo, para ayudar en la decisión de continuar o no.

4.2. La fase de inicio podría ser muy breve

El propósito de la fase de inicio es establecer una visión común inicial de los objetivos del proyecto, determinar si es viable y decidir si merece la pena llevar a cabo algunas investigaciones serias en la fase de elaboración. Si se ha decidido de antemano que el proyecto se hará sin ninguna duda, y es claramente viable (quizás porque el equipo ha desarrollado proyectos parecidos antes), entonces la fase de inicio será especialmente breve. Podría incluir los primeros talleres de requisitos¹, planificación de la primera iteración y, entonces, rápidamente, cambiar a la elaboración.

4.3. ¿Qué artefactos podrían crearse en la fase de inicio?

La Tabla 4.1 presenta un listado de los artefactos comunes de la fase de inicio (o principio de la elaboración) e indica las cuestiones que deben abordarse. Los capítulos siguientes estudiarán algunos de ellos con más detalle, especialmente el Modelo de Ca-

Tabla 4.1. Ejemplo de artefactos de la fase de inicio.

Artefacto†	Comentario
Visión y Análisis del Negocio	Describe los objetivos y las restricciones de alto nivel, el análisis del negocio y proporciona un informe para la toma de decisiones.
Modelo de Casos de Uso	Describe los requisitos funcionales y los no funcionales relacionados.
Especificación Complementaria	Describe otros requisitos.
Glosario	Terminología clave del dominio.
Lista de Riesgos & Plan de Gestión del Riesgo darles respuesta.	Describe los riesgos del negocio, técnicos, recursos, planificación, y las ideas para mitigarlos o
Prototipos y pruebas-de-conceptos	Para clarificar la visión y validar las ideas técnicas.
Plan de Iteración	Describe qué hacer en la primera iteración de la elaboración.
Fase Plan de & Plan de Desarrollo de Software	Estimación de poca precisión de la duración y esfuerzo de la fase de elaboración. Herramientas, personas, formación y otros recursos.
Marco de Desarrollo	Una descripción de los pasos del UP y los artefactos adaptados para este proyecto. El UP siempre se debe adaptar al proyecto.

† Estos artefactos se completan sólo parcialmente en esta fase. Se refinarán de manera iterativa en las siguientes iteraciones. El nombre en mayúsculas indica que es un artefacto UP con ese nombre oficial.

¹ N. del T.: Se ha traducido “requirements workshop” por “taller de requisitos” para indicar una reunión de discusión sobre requisitos.

sos de Uso. Una idea clave con respecto al desarrollo iterativo es comprender que estos artefactos sólo se completan parcialmente en esta fase, se refinarán en iteraciones posteriores, e incluso no deberían crearse a menos que se considere probable que añadirán valor práctico real. Y, puesto que estamos en el inicio, la investigación y el contenido de los artefactos deberían ser ligeros.

Por ejemplo, el Modelo de Casos de Uso (que se describirá en los capítulos siguientes) podría listar los *nombres* de la mayoría de los casos de uso y actores esperados, pero quizás sólo describiría en detalle el 10% de los casos de uso —hecho con el propósito de desarrollar una visión de alto nivel y sin detalles del alcance, objetivo y riesgos del sistema.

Nótese que en la fase de inicio se podrían realizar algunas tareas de programación con el objeto de crear prototipos de “pruebas de conceptos”, para clarificar unos pocos requisitos mediante (generalmente) prototipos orientados a la interfaz de usuario, y hacer experimentos de programación para cuestiones técnicas críticas.

¿No es eso mucha documentación?

Hay que recordar que los artefactos se deberían considerar opcionales. Se deben elegir sólo aquellos que realmente añadan valor al proyecto, y desecharlos si no se prueba que merezcan la pena.

Lo importante de un artefacto no es el documento o el diagrama en sí mismo, sino el pensamiento, análisis y disposición activa (y entonces se registra, para evitar re-inventaciones o tener que repetir las cosas de palabra). Como dijo el general Eisenhower: “Al preparar una batalla siempre he encontrado que los planes no son útiles, pero planificar indispensable” [Nixon90, BF00].

Hay que almacenar los artefactos digitalmente y on-line —disponibles en el sitio web del proyecto— en lugar de en papel.

Obsérvese también que los artefactos del UP de los proyectos anteriores se pueden utilizar en otros posteriores. Es normal que haya muchas similitudes entre los proyectos en cuanto a los artefactos de riesgos, gestión del proyecto, pruebas y entorno. Todos los proyectos UP organizarán (o deberían organizar) los artefactos de la misma manera, con los mismos nombres (Lista de Riesgos, Marco de Desarrollo, etc.). Esto simplifica la búsqueda de artefactos reutilizables de los proyectos precedentes en las nuevas aplicaciones del UP.

4.4. No se entendió la fase de inicio cuando...

- La duración es mayor de “unas pocas” semanas en la mayoría de los proyectos.
- Se intenta definir la mayoría de los requisitos.
- Se espera que los planes y estimaciones sean fiables.
- Se define la arquitectura; en lugar de hacerlo de manera iterativa en la fase de elaboración.

- Se cree que la secuencia adecuada de trabajo debería ser: 1) definición de los requisitos; 2) diseño de la arquitectura; 3) implementación.
- No hay artefacto de Análisis del Negocio o Visión.
- No se identificaron la mayoría de los nombres de los casos de uso y los actores.
- Se escribieron todos los casos de uso en detalle.
- Ninguno de los casos de uso se escribió en detalle; cuando del 10-20% se deberían escribir con detalle para obtener algún conocimiento realista del alcance del problema.

COMPRENSIÓN DE LOS REQUISITOS

*El nuestro es un mundo donde la gente no sabe lo que quiere
y está deseando atravesar el infierno para conseguirlo.*

Don Marquis

Objetivos

- Definir el modelo FURPS+.
 - Relacionar los tipos de requisitos con los artefactos del UP.
-

Introducción

No todos los requisitos se crean igual. Este capítulo presenta la clasificación de requisitos FURPS+.

Los **requisitos** son capacidades y condiciones con las cuales debe ser conforme el sistema —y más ampliamente, el proyecto [JBR99]—. El primer reto del trabajo de los requisitos es encontrar, comunicar y recordar (que normalmente significa registrar) lo que se necesita realmente, de manera que tenga un significado claro para el cliente y los miembros del equipo de desarrollo.

El UP fomenta un conjunto de buenas prácticas, una de las cuales es la *gestión de requisitos*. Esto no hace referencia a la actitud del ciclo de vida en cascada de definir completamente y estabilizar los requisitos en la primera fase del proyecto, sino más bien —en el contexto de que inevitablemente los deseos del personal involucrado son

cambiantes y poco claros— “un enfoque sistemático para encontrar, documentar, organizar y seguir la pista de los requisitos cambiantes de un sistema” [RUP]; en concreto, haciéndolo con destreza y sin ser descuidado. Fíjese en la palabra *cambiantes*; el UP acepta el cambio en los requisitos como un motor fundamental del proyecto. Otro término importante es *encontrar*; es decir, elicitar cuidadosamente mediante técnicas tales como escritura de casos de uso y talleres de requisitos.

Como se indica en la Figura 5.1, un estudio sobre los costes en proyectos reales en diferentes empresas reveló que el 37% de ellos estaban relacionados con los requisitos, de manera que las cuestiones de requisitos constituyen la principal causa de problemas [Standish94]. En consecuencia, es importante adquirir dominio en la gestión de requisitos. La respuesta del ciclo de vida en cascada a este dato sería intentar con ahínco pulir, estabilizar y fijar los requisitos antes de cualquier diseño o implementación, pero la historia demuestra que es una batalla perdida. La respuesta iterativa es utilizar un proceso que acepte el cambio y la retroalimentación como motores centrales en el descubrimiento de los requisitos.

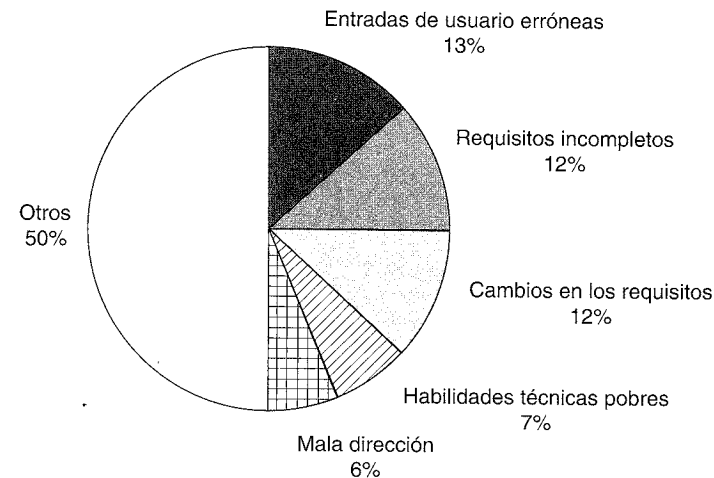


Figura 5.1. Factores del coste en proyectos software reales.

5.1. Tipos de requisitos

En el UP, los requisitos se clasifican de acuerdo con el modelo FURPS+ [Grady92], un útil nemotécnico que significa los siguientes cinco tipos de requisitos¹:

- **Funcional (*Functional*)**: características, capacidades y seguridad.
- **Facilidad de uso (*Usability*)**: factores humanos, ayuda, documentación.
- **Fiabilidad (*Reliability*)**: frecuencia de fallos, capacidad de recuperación de un fallo y grado de previsión.

¹ Hay varios sistemas de clasificación de requisitos y atributos de calidad publicados en libros y por organizaciones estándares, como el ISO 9126 (que es similar a la lista del FURPS+), y varias del Instituto de Ingeniería del Software (SEI, *Software Engineering Institute*); cualquiera de ellas se puede utilizar en un proyecto UP.

- **Rendimiento (*Performance*)**: tiempos de respuesta, productividad, precisión, disponibilidad, uso de los recursos.
- **Soporte (*Supportability*)**: adaptabilidad, facilidad de mantenimiento, internacionalización, configurabilidad.

El ‘+’ en FURPS+ indica requisitos adicionales, tales como:

- **Implementación**: limitación de recursos, lenguajes y herramientas, hardware...
- **Interfaz**: restricciones impuestas para la interacción con sistemas externos.
- **Operaciones**: gestión del sistema en su puesta en marcha.
- **Empaquetamiento**
- **Legales**: licencias, etcétera.

Resulta útil utilizar las categorías del FURPS+ (o algún esquema de clasificación) como una lista para comprobar que se cubren los requisitos, de manera que reducimos el riesgo de no considerar alguna faceta importante del sistema.

Algunos de estos requisitos se denominan colectivamente **atributos de calidad**, **requisitos de calidad**, o las “-ilities”² de un sistema. Éstos comprenden facilidad de uso (*usability*), fiabilidad (*reliability*), rendimiento (*performance*) y soporte (*supportability*). Lo normal es dividir los requisitos en **funcionales** (comportamiento) y **no funcionales** (todo lo demás); a algunos no les gusta esta amplia generalización [BCK98], pero se utiliza de manera muy extendida.

Los requisitos funcionales se estudian y recogen en el Modelo de Casos de Uso, el tema del siguiente capítulo, y en la lista de características del sistema del artefacto Visión. Los otros requisitos se pueden recoger en los casos de usos con los que están relacionados, o en el artefacto Especificación Complementaria. El artefacto Visión resume los requisitos de alto nivel que se elaboran en estos otros documentos. El Glosario agrupa y clarifica los términos que se utilizan en los requisitos. El Glosario en el UP también comprende el concepto de **diccionario de datos**, que reúne los requisitos relacionados con los datos, como reglas de validación, valores aceptables, etcétera. Los prototipos son un mecanismo para clarificar qué es lo que se quiere o es posible.

Como veremos cuando estudiemos el análisis arquitectural, los requisitos de calidad influyen fuertemente en la arquitectura de un sistema. Por ejemplo, un requisito de alto rendimiento y alta fiabilidad influirá en la elección de componentes software y hardware, y en sus configuraciones. La necesidad de fácil adaptación, debido a cambios frecuentes en los requisitos funcionales, igualmente dará forma al diseño del software.

5.2. Lecturas adicionales

En los capítulos siguientes se cubrirán referencias relacionadas con casos de uso. Se recomiendan como punto de partida para el estudio de los requisitos, textos sobre requisitos orientados a los casos de uso, como *Writing Effective Use Cases* [Cockburn01], en lugar de textos más generales (y normalmente, tradicionales).

² N. del T.: Terminación plural en inglés de los nombres de tales requisitos.

Hay un amplio debate sobre requisitos —y una extensa variedad de temas de ingeniería del software— bajo el paraguas de Cuerpo de Conocimientos en Ingeniería del Software (**SWEBOK**, *Software Engineering Body of Knowledge*) disponible en www.swebok.org.

El SEI (www.sei.cmu.edu) tiene varias propuestas relacionadas con los requisitos de calidad. El ISO 9126, el IEEE Std. 830, y el IEEE Std. 1061 son estándares relacionados con los requisitos y atributos de calidad, y están disponibles en la web en varios sitios.

Algunas advertencias con respecto a los libros generales sobre requisitos, incluso aquellos que proponen cubrir los casos de uso, desarrollo iterativo o, de hecho, incluso los requisitos en el UP:

1. La mayoría están escritos bajo la influencia de un ciclo de vida en cascada planteando la definición “precisa” de los requisitos por adelantado, antes de pasar al diseño y la implementación. Esto no pretende invalidar su valor más amplio o, a menudo, profundas y útiles visiones de los requisitos independientes del método, sino que pretende aclarar que no constituyen una visión fiable del desarrollo iterativo. Esto es debido a que la experiencia de los autores proviene principalmente de proyectos de ciclo de vida en cascada, trabajando para refinar, definir cuidadosamente y precisamente los requisitos, y terminar con la fase de requisitos antes de continuar con el diseño. Aquellos libros que también mencionan el desarrollo iterativo, lo abordan muy superficialmente, quizás añadiendo material “iterativo” apelando a las tendencias modernas. Por tanto, los libros y artículos de requisitos deberían leerse con cautela; uno podría sentirse seguro con la idea de intentar definir de manera cuidadosa todos los requisitos en la fase inicial, que no es consistente con un proceso iterativo.
2. Muchos libros generales sobre requisitos que también dicen incluir casos de uso, lo hacen muy superficialmente, o entienden mal el significado real de requisitos dirigidos por casos de uso. Esto podría deberse a que los autores poseen una experiencia de muchos años en métodos de requisitos tradicionales, y recientemente, han intentado incluir los casos de uso en sus métodos anteriores, sin darse cuenta de que la idea principal de los casos de uso, tal y como la concibe Ivar Jacobson y el UP, es hacer los casos de uso el elemento central del enfoque de requisitos global —sustituyendo a otros documentos de requisitos como elemento central—; los casos de uso impregnan y dirigen el trabajo de requisitos, en lugar de considerarse como una técnica de apoyo auxiliar a nivel inferior o medio, añadida a los documentos o enfoques de requisitos tradicionales.

En resumen, los libros generales sobre requisitos ofrecen consejos útiles sobre técnicas y cuestiones para recopilar los requisitos, escritos por personas expertas, pero normalmente, los presentan en el contexto de un proceso en cascada, sin tener un gran conocimiento de las implicaciones más profundas de los casos de uso. Cualquier consejo sobre el proceso que sea una variante de: “trate de definir la mayoría de los requisitos, y entonces pase al diseño y la implementación” no es consistente con el desarrollo iterativo y el UP.

MODELO DE CASOS DE USO: ESCRITURA DE REQUISITOS EN CONTEXTO

El primer paso indispensable para conseguir las cosas que quieres de la vida: decidir qué quieres.
Ben Stein

Objetivos

- Identificar y escribir casos de uso.
- Relacionar los casos de uso con los objetivos de los usuarios y los procesos de negocio básicos.
- Utilizar los formatos breve, informal y completo, en un estilo esencial.
- Relacionar el trabajo con casos de uso con el desarrollo iterativo.

Introducción

Merece la pena estudiar este capítulo durante la primera lectura del libro porque los casos de uso son un mecanismo ampliamente utilizado para descubrir y registrar los requisitos (especialmente los funcionales); influyen muchos aspectos de un proyecto, incluyendo el A/DOO. Merece la pena tanto saber sobre los casos de uso como crearlos.

La escritura de casos de uso —historias del uso de un sistema— es una técnica excelente para entender y describir los requisitos. Este capítulo explora los conceptos claves de los casos de uso y presenta casos de uso de ejemplo para la aplicación NuevaEra.

El UP define el **Modelo de Casos de Uso** en la disciplina Requisitos. Básicamente, es el conjunto de todos los casos de uso; es un modelo de la funcionalidad y entorno del sistema.

6.1. Objetivos e historias

Los clientes y los usuarios finales tienen objetivos (también conocidos como *necesidades*) y quieren sistemas informáticos que les ayuden a conseguirlos, que varían desde registrar las ventas hasta estimar el flujo de petróleo de futuros pozos. Hay varias formas de capturar estos objetivos y requisitos del sistema; las mejores son simples y familiares, porque esto hace que sea más fácil —especialmente para clientes y usuarios finales— contribuir a su definición o evaluación. Eso reduce el riesgo de perder el hilo.

Los casos de uso son un mecanismo para ayudar a mantenerlo simple y entendible para todo el personal involucrado. De manera informal, son historias del uso de un sistema para alcanzar los objetivos. A continuación presentamos un ejemplo de caso de uso en *formato breve*:

Procesar Venta: Un cliente llega a una caja con artículos para comprar. El cajero utiliza el sistema PDV para registrar cada artículo comprado. El sistema presenta una suma parcial y detalles de cada línea de venta. El cliente introduce los datos del pago, que el sistema valida y registra. El sistema actualiza el inventario. El cliente recibe un recibo del sistema y luego se va con los artículos.

A menudo, los casos de uso necesitan una elaboración mayor que ésta, pero la esencia es descubrir y registrar los requisitos funcionales, mediante la escritura de historias del uso de un sistema, para ayudar a cumplir los objetivos de varias de las personas involucradas; esto es, los *casos de uso*¹. Se supone que no es una idea difícil, aunque, de hecho podría ser difícil descubrir o decidir lo que es necesario, y escribirlo de manera coherente con un nivel de detalle útil.

Se ha escrito mucho acerca de los casos de uso, y si bien es útil, existe el riesgo entre las personas inteligentes y creativas, de oscurecer una idea sencilla con niveles de sofisticación. Normalmente es posible distinguir a un modelador de casos de uso novato (o a un analista serio de Tipo A) preocupándose en exceso con cuestiones secundarias como diagramas de casos de uso, relaciones de casos de uso, paquetes de casos de uso, atributos opcionales, etcétera, en lugar de escribir las historias. En otras palabras, el poder del mecanismo de casos de uso es la capacidad tanto de aumentar como de disminuir, en términos de sofisticación y formalidad, dependiendo de la necesidad.

6.2. Antecedentes

La idea de utilizar los casos de uso para describir los requisitos funcionales fue introducida en 1986 por Ivar Jacobson [Jacobson92], uno de los contribuidores principales al UML y UP. La idea de caso de uso de Jacobson ha tenido una gran influencia y ha sido ampliamente reconocida; siendo sus principales virtudes la simplicidad y utilidad. Aunque muchos han contribuido en este campo, se puede sostener que el siguiente paso más coherente, comprensible e influyente en la definición de qué son (o deberían ser) los casos de uso y cómo escribirlos, procede de Alistair Cockburn, resumido en su popular texto *Writing Effective Use Cases* [Cockburn01], basado en sus primeros trabajos y escritos publicados de 1992 en adelante. Esta introducción, por tanto, se basa y es consistente con este último trabajo.

¹ El término original en sueco se traduce literalmente como “caso de costumbre”.

6.3. Casos de uso y valor añadido

En primer lugar, algunas definiciones informales: un **actor** es algo con comportamiento, como una persona (identificada por un rol), sistema informatizado u organización; por ejemplo, un cajero.

Un **escenario** es una secuencia específica de acciones e interacciones entre los actores y el sistema objeto de estudio; también se denomina **instancia de caso de uso**. Es una historia particular del uso de un sistema, o un camino a través del caso de uso; por ejemplo, el escenario de éxito de compra de artículos con pago en efectivo, o el escenario de fallo al comprar debido al rechazo de la transacción de pago con la tarjeta de crédito.

Informalmente entonces, un **caso de uso** es una colección de escenarios con éxito y fallo relacionados, que describe a los actores utilizando un sistema para satisfacer un objetivo. Por ejemplo, a continuación presentamos un caso de uso en *formato informal* que incluye algunos escenarios alternativos:

Gestionar Devoluciones

Escenario principal de éxito: Un cliente llega a una caja con artículos para devolver. El cajero utiliza el sistema PDV para registrar cada uno de los artículos devueltos...

Escenarios alternativos:

Si se pagó con tarjeta de crédito, y se rechaza la transacción de reembolso a su cuenta, informar al cliente y pagarle en efectivo.

Si el identificador del artículo no se encuentra en el sistema, notificar al Cajero y sugerir la entrada manual del código de identificación (quizás esté alterado).

Si el sistema detecta fallos en la comunicación con el sistema de contabilidad externo...

El RUP proporciona una definición alternativa, aunque similar, de un caso de uso:

Un conjunto de instancias de caso de uso, donde cada instancia es una secuencia de acciones que un sistema ejecuta, produciendo un resultado observable de valor para un actor particular [RUP].

La expresión “*un resultado observable de valor*” es sutil pero importante, porque destaca el hecho de que el comportamiento del sistema debería preocuparse de proporcionar valor al usuario.

Una actitud clave en el trabajo con casos de uso es centrarse en la pregunta: “¿Cómo puedo, utilizando el sistema, proporcionar un valor observable al usuario, o cumplir sus objetivos?” en lugar de, simplemente, pensar en los requisitos del sistema en términos de una “lista de la lavandería” de características o funciones.

Quizás parece obvio destacar que se proporcione un valor observable para el usuario, pero la industria de software está plagada de proyectos fracasados que no proporciona-

ron lo que la gente realmente necesitaba. El enfoque de la lista de características y funciones para capturar los requisitos, puede contribuir a este resultado negativo, puesto que no fomenta que el personal involucrado considere los requisitos en un contexto amplio de uso del sistema, en un escenario para alcanzar algún resultado observable de valor, o algún objetivo. Por el contrario, los casos de uso sitúan las características y funciones en un contexto orientado al objetivo. De ahí el título del capítulo².

Ésta es la idea clave que Jacobson intentaba transmitir con el concepto de caso de uso: trabaja con los requisitos centrándote en cómo puede un sistema añadir valor y cumplir los objetivos.

6.4. Casos de uso y requisitos funcionales

Los casos de uso son requisitos; ante todo son requisitos funcionales que indican qué hará el sistema. En términos de los tipos de requisitos FURPS+, los casos de uso se refieren fundamentalmente a la “F” (funcional o de comportamiento), pero también pueden utilizarse para otros tipos, especialmente cuando esos otros tipos están estrechamente relacionados con un caso de uso. En el UP —y métodos más modernos— los casos de uso son el mecanismo principal que se recomienda para su descubrimiento y definición. Los casos de uso definen una promesa o contrato de la manera en la que se comportará un sistema.

Para ser claros: los casos de uso *son* requisitos (aunque no todos los requisitos). Algunos piensan en requisitos sólo como listas de características y funciones de la forma “el sistema deberá hacer...”. No es así, y una idea clave de los casos de uso es (por lo general) reducir la importancia o el uso de listas de características detalladas al estilo antiguo y más bien, escribir casos de uso para los requisitos funcionales. Veremos más sobre este punto en una sección posterior.

Los casos de uso son documentos de texto, no diagramas, y el modelado de casos de uso es, sobre todo, una acción de escribir texto, no dibujar. Sin embargo, UML define un diagrama de casos de uso para ilustrar los nombres de casos de uso y actores, y sus relaciones.

6.5. Tipos de casos de uso y formatos

Casos de uso de caja negra y las responsabilidades del sistema

Los casos de uso de **caja negra** son la clase más común y recomendada; no describen el funcionamiento interno del sistema, sus componentes o diseño, sino que se describe el sistema en base a las *responsabilidades* que tiene, que es una metáfora común y unificadora en el pensamiento orientado a objetos —los elementos software tienen responsabilidades y colaboran con otros elementos que tienen responsabilidades—.

² Procede del libro titulado muy apropiadamente *Uses Cases: Requirements in Context* [GK00] (el título del capítulo se adaptó con permiso de los autores).

A través de la definición de las responsabilidades del sistema con casos de uso de caja negra, es posible especificar *qué* debe hacer el sistema (los requisitos funcionales) sin decidir *cómo* lo hará (el diseño). De hecho, la definición de “análisis” frente al “diseño” se resume algunas veces como el “qué” frente al “cómo”. Éste es un tema importante en un buen desarrollo de software: evite durante el análisis de requisitos tomar decisiones acerca del “cómo”, y especifique el comportamiento externo del sistema, como una caja negra. Después, durante el diseño, cree una solución que satisfaga la especificación.

Estilo de caja negra	No
El sistema registra la venta	El sistema escribe la venta en una base de datos... o (incluso peor). El sistema genera una sentencia SQL INSERT para la venta...

Tipos de formalidad

Los casos de uso se escriben con formatos diferentes, dependiendo de la necesidad. Además del tipo de *visibilidad*, de caja negra frente a caja blanca, los casos de uso se escriben con varios grados de *formalidad*:

- **Breve:** resumen conciso de un párrafo, normalmente del escenario principal con éxito. El anterior ejemplo de *Procesar Venta* era breve.
- **Informal:** formato de párrafo en un estilo informal. Múltiples párrafos que comprenden varios escenarios. El anterior ejemplo de *Gestionar Devoluciones* era informal.
- **Completo:** el más elaborado. Se escriben con detalle todos los pasos y variaciones, y hay secciones de apoyo como precondiciones y garantías de éxito.

El ejemplo siguiente es un caso de uso en formato completo del caso de estudio NuevaEra.

6.6. Ejemplo completo: Procesar Venta

Los casos de uso completos muestran más detalles y están estructurados; son útiles para entender en profundidad los objetivos, tareas y requisitos. En el caso de estudio del PDV NuevaEra, se crearían en uno de los primeros talleres de requisitos con la colaboración del analista del sistema, expertos en la materia de estudio y los desarrolladores.

El formato usecases.org

Hay varias plantillas disponibles para los casos de uso completos. Sin embargo, quizás el formato más ampliamente extendido y compartido es la plantilla disponible en www.usecases.org. El siguiente ejemplo muestra este estilo.

El lector debe darse cuenta que éste es el principal ejemplo de caso de uso detallado para el caso de estudio del libro; muestra muchos elementos y cuestiones comunes.

Caso de uso UC1: Procesar Venta

Actor principal: Cajero.

Personal involucrado e intereses:

- Cajero: quiere entradas precisas, rápidas, y sin errores de pago, ya que las pérdidas se deducen de su salario.
- Vendedor: quiere que las comisiones de las ventas estén actualizadas.
- Compañía: Quiere registrar las transacciones con precisión y satisfacer los intereses de los clientes. Quiere asegurar que se registran los pagos aceptados por el Servicio de Autorización de Pagos. Quiere cierta tolerancia a fallos que permita capturar las ventas incluso si los componentes del servidor (ej. validación remota de crédito) no están disponibles. Quiere actualización automática y rápida de la contabilidad y el inventario.
- Agencia Tributaria del Gobierno: quiere recopilar los impuestos de cada venta. Podrían ser múltiples agencias: nacional, provincial y local.
- Servicio de Autorización de Pagos: Quiere recibir peticiones de autorización digital con el formato y protocolo correctos. Quiere registrar de manera precisa las cuentas por cobrar de la tienda.

Precondiciones: El cajero se identifica y autentica.

Garantías de éxito (Postcondiciones): Se registra la venta. El impuesto se calcula de manera correcta. Se actualizan la contabilidad y el inventario. Se registran las comisiones. Se genera el recibo. Se registran las autorizaciones de pago aprobadas.

Escenario principal de éxito (o Flujo Básico):

1. El Cliente llega a un terminal PDV con mercancías y/o servicios que comprar.
2. El Cajero comienza una nueva venta.
3. El Cajero introduce el identificador del artículo.
4. El Sistema registra la línea de la venta y presenta la descripción del artículo, precio y suma parcial. El precio se calcula a partir de un conjunto de reglas de precios.

El Cajero repite los pasos 3-4 hasta que se indique.

5. El Sistema presenta el total con los impuestos calculados.
6. El Cajero le dice al Cliente el total y pide que le pague.
7. El Cliente paga y el Sistema gestiona el pago.
8. El Sistema registra la venta completa y envía la información de la venta y el pago al sistema de Contabilidad externo (para la contabilidad y las comisiones) y al sistema de Inventario (para actualizar el inventario).
9. El Sistema presenta el recibo.
10. El Cliente se va con el recibo y las mercancías (si es el caso).

Extensiones (o Flujos Alternativos):

*a. En cualquier momento el Sistema falla:

Para dar soporte a la recuperación y registro correcto, asegura que todos los estados y eventos significativos de una transacción puedan recuperarse desde cualquier paso del escenario.

1. El Cajero reinicia el Sistema, inicia la sesión, y solicita la recuperación al estado anterior.
2. El Sistema reconstruye el estado anterior.
 - 2a. El Sistema detecta anomalías intentando la recuperación:
 1. El Sistema informa del error al Cajero, registra el error, y pasa a un estado limpio.
 2. El Cajero comienza una nueva venta.

3a. Identificador no válido:

1. El Sistema señala el error y rechaza la entrada.

3b. Hay muchos artículos de la misma categoría y tener en cuenta una única identidad del artículo no es importante (ej. 5 paquetes de hamburguesas vegetales):

1. El Cajero puede introducir el identificador de la categoría del artículo y la cantidad.

3-6a. El Cliente le pide al Cajero que elimine un artículo de la compra:

1. El Cajero introduce el identificador del artículo para eliminarlo de la compra.
2. El Sistema muestra la suma parcial actualizada.

3-6b. El Cliente le pide al Cajero que cancele la venta:

1. El Cajero cancela la venta en el Sistema.

3-6c. El Cajero detiene la venta:

1. El sistema registra la venta para que esté disponible su recuperación en cualquier terminal PDV.

4a. El Sistema genera el precio de un artículo que no es el deseado (ej. el Cliente se queja por algo y se le ofrece un precio más bajo):

1. El Cajero introduce el precio alternativo.
2. El Sistema presenta el precio nuevo.

5a. El sistema encuentra algún fallo para comunicarse con el servicio externo del sistema de cálculo de impuestos.

1. El Sistema reinicia el servicio en el nodo PDV y continúa.

1a. El Sistema detecta que el servicio no se reinicia.

1. El Sistema señala el error.
2. El Cajero podría calcular e introducir manualmente el impuesto, o cancelar la venta.

5b. El Cliente dice que le son aplicables descuentos (ej. empleado, cliente preferente):

1. El Cajero señala la petición de descuento.
2. El Cajero introduce la identificación del Cliente.
3. El Sistema presenta el descuento total, basado en las reglas de descuento.

5c. El Cliente dice que tiene crédito en su cuenta, para aplicar a la venta:

1. El Cajero señala la petición de crédito.
2. El Cajero introduce la identificación del Cliente.
3. El Sistema aplica el crédito hasta que el precio = 0, y reduce el crédito que queda.

6a. El Cliente dice que su intención era pagar en efectivo pero que no tiene suficiente:

- 1a. El Cliente utiliza un método de pago alternativo.
- 1b. El Cliente le dice al Cajero que cancele la venta. El Cajero cancela la venta en el Sistema.

7a. Pago en efectivo:

1. El Cajero introduce la cantidad de dinero en efectivo entregada.
2. El Sistema muestra la cantidad de dinero a devolver y abre el cajón de caja.
3. El Cajero deposita el dinero entregado y devuelve el cambio al Cliente.
4. El Sistema registra el pago en efectivo.

7b. Pago a crédito:

1. El Cliente introduce la información de su cuenta de crédito.
2. El Sistema envía la petición de autorización del pago al Sistema externo de Servicio de Autorización de Pagos, y solicita la aprobación del pago.
 - 2a. El Sistema detecta un fallo en la colaboración con el sistema externo:
 1. El Sistema señala el error al Cajero.
 2. El Cajero le pide al Cliente un modo de pago alternativo.
3. El Sistema recibe la aprobación del pago y lo notifica al Cajero.
 - 3a. El Sistema recibe la denegación del pago:
 1. El Sistema señala la denegación al Cajero.
 2. El Cajero le pide al Cliente un modo de pago alternativo.

- 4. El Sistema registra el pago a crédito, que incluye la aprobación del pago
- 5. El Sistema presenta el mecanismo de entrada para la firma del pago a crédito.
- 6. El Cajero le pide al Cliente que firme el pago a crédito. El Cliente introduce la firma.
- 7c. Pago con cheque...
- 7d. Pago a cuenta...
- 7e. El Cliente presenta cupones:
 - 1. Antes de gestionar el pago, el Cajero recoge cada cupón y el Sistema reduce el pago como sea oportuno. El sistema registra los cupones utilizados por razones de contabilidad.
 - 1a. El cupón introducido no es válido para ninguno de los artículos comprados
 - 1. El Sistema señala el error al Cajero.
- 9a. Hay rebajas en los artículos:
 - 1. El Sistema presenta los formularios de rebaja y los recibos de descuento para cada artículo con una rebaja.
- 9b. El Cliente solicita un vale-regalo (sin precio visible):
 - 1. El Cajero solicita el vale-regalo y el Sistema lo proporciona.

Requisitos especiales:

- Interfaz de Usuario con pantalla táctil en un gran monitor de pantalla plana. El texto debe ser visible a un metro de distancia.
- Tiempo de respuesta para la autorización de crédito de 30 segundos el 90% de las veces.
- De algún modo, queremos recuperación robusta cuando falla el acceso a servicios remotos, como el sistema de inventario.
- Internacionalización del lenguaje del texto que se muestra.
- Reglas de negocio que se puedan añadir en tiempo de ejecución en los pasos 3 y 7.
- ...

Lista de tecnología y variaciones de datos:

- 3a. El identificador del artículo se introduce mediante un escáner láser de código de barras (si está presente el código de barras) o a través del teclado.
- 3b. El identificador del artículo podría ser cualquier esquema de código UPC, EAN, JAN o SKU.
- 7a. La entrada de la información de la cuenta de crédito se lleva a cabo mediante un lector de tarjetas o el teclado.
- 7b. La firma de los pagos a crédito se captura en un recibo de papel. Pero en dos años, pronosticamos que muchos clientes querrán que se capture la firma digital.

Frecuencia: Podría ser casi continuo.

Temas abiertos:

- ¿Cuáles son las variaciones de la ley de impuestos?
- Explorar las cuestiones de recuperación de servicios remotos.
- ¿Cuál es la adaptación que se tiene que hacer para diferentes negocios?
- ¿Un cajero debe llevarse el dinero de la caja cuando salga del sistema?
- ¿Puede utilizar el cliente directamente el lector de tarjetas o tiene que hacerlo el cajero?

Este caso de uso es más bien ilustrativo que exhaustivo (aunque está basado en los requisitos de un sistema PDV real). Sin embargo, muestra suficiente detalle y complejidad para exponer de manera realista que los casos de uso completos pueden documentar muchos detalles de los requisitos. Este ejemplo nos servirá bien como modelo para muchos problemas de los casos de uso.

La variación de dos-columnas

Algunos prefieren el formato en dos columnas o conversacional, que destaca el hecho de que se establece una interacción entre los actores y el sistema. Se propuso por primera vez por Rebecca Wirfs-Brock en [Wirfs-Brock93], y también promueven su uso Constantine y Lockwood para ayudar en el análisis e ingeniería de usabilidad [CL99]. A continuación presentamos el mismo contenido utilizando el formato en dos columnas:

Caso de uso UC1: Procesar Venta

Actor principal:como antes...	
Escenario principal de éxito:	
Acción del actor (o intención)	Responsabilidad del Sistema
1. El Cliente llega a un terminal PDV con mercancías y/o servicios que comprar.	
2. El Cajero comienza una nueva venta.	
3. El Cajero introduce el identificador del artículo.	4. Registra cada línea de la venta y presenta la descripción del artículo y la suma parcial.
<i>El Cajero repite los pasos 3-4 hasta que se indique</i>	
6. El Cajero le dice al Cliente el total y pide que le pague.	5. El Sistema presenta el total con los impuestos calculados.
7. El Cliente paga.	8. El Sistema gestiona el pago.
	9. Registra la venta completa y envía la información de la venta y el pago al sistema de Contabilidad externo (para la contabilidad y las comisiones) y al sistema de Inventario (para actualizar el inventario). El Sistema presenta el recibo.
...	...

¿Cuál es el mejor formato?

No existe un mejor formato; unos prefieren el estilo de una columna, otros el de dos columnas. Las secciones se pueden añadir y quitar; los nombres de los títulos podrían cambiar. Ninguna de estas cosas es especialmente importante; la clave es escribir los detalles del escenario principal de éxito y sus extensiones de alguna forma. [Cockburn1] resume muchos formatos utilizables:

Práctica personal

Ésta es mi práctica, no una recomendación. Durante algunos años, utilicé el formato en dos columnas debido a su clara separación visual de la conversación. Sin embargo, he vuelto al estilo en una columna ya que es más compacto y más fácil de formatear, y el pequeño valor de la separación visual de la conversación, para mí, no supera estos beneficios. Encuentro que es todavía sencillo identificar visualmente las diferentes partes en la conversación (Cliente, Sistema,...) si cada parte y las respuestas del Sistema se asignan normalmente a sus propios pasos.

6.7. Explicación de las secciones

Elementos del prólogo

Son posibles muchos elementos opcionales en el prólogo. Sitúe al principio sólo los elementos que son importantes que se lean antes del escenario principal de éxito. Mueva el material de “encabezamiento” ajeno, al final de los casos de uso.

Actor principal: El actor principal que recurre a los servicios del sistema para cumplir un objetivo.

Importante: Personal involucrado y lista de intereses

Esta lista es más importante y práctica de lo que podría parecer a primera vista. Sugiere y delimita qué es lo que debe hacer el sistema. Citando a Cockburn:

El [sistema] funciona siguiendo un contrato entre el personal involucrado, donde los casos de usos detallan la parte de comportamiento del contrato... El caso de uso, como contrato de comportamiento, captura *todo y sólo* el comportamiento relacionado con la satisfacción de los intereses del personal involucrado [Cockburn01].

Esto contesta la pregunta: ¿Qué debería estar en un caso de uso? La respuesta es: lo que satisfaga los intereses de todo el personal involucrado. Además, empezando con el personal involucrado y sus intereses antes de escribir el resto del caso de uso, tenemos un modo de recordarnos cuáles deberían ser las responsabilidades más detalladas del sistema. Por ejemplo, ¿habría identificado una responsabilidad para gestionar la comisión del vendedor si no hubiese listado en primer lugar la persona involucrada vendedor y sus intereses? En el mejor de los casos al final, pero quizás lo habría echado en falta durante la primera sesión de análisis. El punto de vista del interés del personal involucrado proporciona un procedimiento metódico y completo para el descubrimiento y registro de todos los comportamientos requeridos.

Personal involucrado e intereses:

- Cajero: quiere entradas precisas, rápidas, y sin errores de pago, ya que las pérdidas se deducen de su salario.
- Vendedor: quiere que las comisiones de las ventas estén actualizadas.
- ...

Precondiciones y garantías de éxito (postcondiciones)

Las **precondiciones** establecen lo que *siempre debe* cumplirse antes de comenzar un escenario en el caso de uso. Las precondiciones *no* se prueban en el caso de uso, sino que son condiciones que se asumen que son verdad. Normalmente, una precondición implica un escenario de otro caso de uso que se ha completado con éxito, como inicio de sesión o el más general “el cajero se identifica y autentica”. Nótese que hay condiciones

que deben ser verdad, pero no tienen un valor práctico para que se escriban, como “el sistema tiene energía”. Las precondiciones comunican suposiciones importantes de las que el escritor del caso de uso piensa que los lectores deberían ser avisados.

Las **garantías de éxito** (o **postcondiciones**) establecen qué debe cumplirse cuando el caso de uso se completa con éxito —o bien el escenario principal de éxito o algún camino alternativo—. La garantía debería satisfacer las necesidades de todo el personal involucrado.

Precondiciones: El cajero se identifica y autentica.

Garantías de éxito (Postcondiciones): Se registra la venta. El impuesto se calcula de manera correcta. Se actualizan la Contabilidad y el Inventario. Se registran las comisiones. Se genera el recibo.

Escenario principal de éxito y pasos (o Flujo Básico)

También recibe el nombre de escenario del “camino feliz”, o más prosaico “Flujo Básico”. Describe el camino de éxito típico que satisface los intereses del personal involucrado. Nótese que, a menudo, *no* incluye ninguna condición o bifurcación. Aunque no es incorrecto o ilegal, se puede suponer que es más comprensible y extensible ser muy consistente, y postergar todo el manejo de caminos condicionales a la sección Extensiones.

Sugerencia

Posponga todas las sentencias condicionales y de bifurcación a la sección Extensiones.

El escenario recoge los pasos, que pueden ser de tres tipos:

1. Una interacción entre actores³.
2. Una validación (normalmente a cargo del sistema).
3. Un cambio de estado realizado por el sistema (por ejemplo, registrando o modificando algo).

El primer paso de un caso de uso, normalmente no está incluido en esta clasificación, sino que indica el evento que desencadena el comienzo del escenario.

Un estilo habitual es poner con mayúsculas los nombres de los actores para facilitar la identificación. También podemos observar el estilo que se utiliza para indicar una repetición.

Escenario principal de éxito (o Flujo Básico):

1. El Cliente llega a un terminal PDV con mercancías para comprar.
 2. El Cajero comienza una nueva venta.
 3. El Cajero introduce el identificador del artículo.
 4. ...
- El Cajero repite los pasos 3-4 hasta que se indique.
5. ...

³ Nótese que el sistema que se está estudiando en sí mismo debería considerarse un actor cuando juega un rol de actor colaborando con otros sistemas.

Extensiones (o Flujos Alternativos)

Las extensiones son muy importantes. Indican todos los otros escenarios o bifurcaciones, tanto de éxito como de fracaso. Podemos observar que en el ejemplo de caso de uso completo, la sección Extensiones es considerablemente más larga y compleja que la correspondiente al Escenario Principal de Éxito; esto es normal y de esperar. También se conocen como “Flujos Alternativos”.

En la escritura de casos de uso completos, la combinación del camino feliz y los escenarios de extensión deberían satisfacer “casi” todos los intereses del personal involucrado. Este punto está limitado, puesto que algunos intereses se podrían capturar mejor como requisitos no funcionales escritos en la Especificación Complementaria en lugar de en los casos de uso.

Los escenarios de extensión son bifurcaciones del escenario principal de éxito y, por tanto, pueden ser etiquetados de acuerdo con él. Por ejemplo, en el Paso 3 del escenario principal podría haber un identificador de artículo inválido, bien porque no se introdujo correctamente o bien porque el sistema no lo conoce. Una extensión se etiqueta como “3a”; primero identifica la condición y después la respuesta. Una extensión alternativa al Paso 3 se etiqueta como “3b” y así sucesivamente.

Extensiones (o Flujos Alternativos):
3a. Identificador no válido:
1. El Sistema señala el error y rechaza la entrada.
3b. Hay muchos artículos de misma categoría y tener en cuenta una única identidad del artículo no es importante (ej. 5 paquetes de hamburguesas vegetales):
1. El Cajero puede introducir el identificador de la categoría del artículo y la cantidad.

Una extensión tiene dos partes: la condición y el manejo.

Guía: Escriba la condición como algo que pueda ser detectado por el sistema o un actor. Para contrastar:

5a. El Sistema detecta un fallo en la comunicación con el servicio externo del sistema de cálculo de impuestos:
5a. El sistema de cálculo de impuestos externo no funciona:

Se prefiere el primer estilo porque se trata de algo que el sistema puede detectar; el último es una inferencia.

El manejo de la extensión se puede resumir en un paso, o incluir una secuencia, como en este ejemplo, que también ilustra la notación utilizada para indicar que una condición puede tener lugar en una serie de pasos:

3-6a. El Cliente le pide al Cajero que elimine un artículo de la compra:
1. El Cajero introduce el identificador del artículo para eliminarlo de la compra.
2. El Sistema muestra la suma parcial actualizada.

Al final del manejo de la extensión, por defecto, el escenario se une de nuevo con el escenario principal de éxito, a menos que la extensión indique otra cosa (como interrumpir el sistema).

Algunas veces, un punto de extensión particular es bastante complejo, como en la extensión “pago a crédito”. Esto puede ser un motivo para expresar la extensión como un caso de uso aparte.

Este ejemplo de extensión también muestra la notación para expresar fallos en las extensiones.

7b. Pago a crédito:
1. El Cliente introduce la información de su cuenta de crédito.
2. El Sistema envía la petición de autorización del pago al Sistema externo de Servicio de Autorización de Pagos, y solicita la aprobación del pago.
2a. El Sistema detecta un fallo en la colaboración con el sistema externo:
1. El Sistema señala el error al Cajero.
2. El Cajero le pide al Cliente un modo de pago alternativo.
3 ...

Si es deseable describir una condición de extensión que puede ser posible durante cualquiera (o al menos la mayoría) de los pasos, se pueden utilizar las etiquetas *a, *b,...

*a. En cualquier momento el Sistema falla:
Para dar soporte a la recuperación y registro correcto, asegura que todos los estados y eventos significativos de una transacción puedan recuperarse desde cualquier paso del escenario.
1. El Cajero reinicia el Sistema, inicia la sesión, y solicita la recuperación al estado anterior.
2. El Sistema reconstruye el estado anterior.

Requisitos especiales

Si un requisito no funcional, atributo de calidad o restricción se relaciona de manera específica con un caso de uso, se recoge en el caso de uso. Esto incluye cualidades tales como rendimiento, fiabilidad y facilidad de uso, y restricciones de diseño (a menudo, en dispositivos de entrada/salida) que son obligados o se consideran probables.

Requisitos especiales:
• Interfaz de usuario con pantalla táctil en un gran monitor de pantalla plana. El texto debe ser visible a un metro de distancia.
• Tiempo de respuesta para la autorización de crédito de 30 segundos el 90% de las veces.
• Internacionalización del lenguaje del texto que se muestra.
• Reglas de negocio que se puedan añadir en ejecución en los pasos 3 y 7.

Un consejo clásico del UP es registrar estos requisitos con el caso de uso, y constituye un lugar razonable al escribir primero el caso de uso. Sin embargo, muchos expertos encuentran útil reunir al final todos los requisitos no funcionales en la Especificación Complementaria, para favorecer la gestión del contenido, comprensión y legibilidad por-

que, normalmente, se tienen que considerar estos requisitos en conjunto durante el análisis arquitectural.

Lista de tecnología y variaciones de datos

A menudo, encontramos variaciones técnicas en cómo se debe hacer algo, pero no en qué, y es importante registrarlo en el caso de uso. Un ejemplo típico es una restricción técnica impuesta por el personal involucrado con respecto a las tecnologías de entrada o salida de datos. Por ejemplo, uno de los interesados podría decir, “El sistema PDV debe soportar la entrada de la cuenta de crédito utilizando un lector de tarjetas y el teclado”. Nótese que éstos son ejemplos de decisiones o restricciones de diseño anticipadas; en general, deben evitarse decisiones de diseño prematuras, pero algunas veces son obvias o inevitables, especialmente en relación con las tecnologías de entrada/salida.

También es necesario entender las variaciones en los esquemas de los datos, como utilizar UPCs o EANs para los identificadores de los artículos, codificados mediante el código de barras.

Esta lista es el lugar para situar tales variaciones. También es útil registrar las variaciones en los datos que podrían capturarse en un paso particular.

Lista de tecnología y variaciones de datos

3a. El identificador del artículo se introduce mediante un escáner láser de código de barras (si está presente el código de barras) o a través del teclado.

3b. El identificador del artículo podría ser cualquier esquema de código UPC, EAN, JAN o SKU.

7a. La entrada de la información de la cuenta de crédito se lleva a cabo mediante un lector de tarjetas o el teclado.

7b. La firma de los pagos a crédito se captura en un recibo de papel. Pero en dos años, pronosticamos que muchos clientes querrán que se capture la firma digital.

Sugerencia

Esta sección no debería contener múltiples pasos para representar la variación de comportamiento en diferentes casos. Si es necesario, dígalos en la sección Extensiones.

6.8. Objetivos y alcance de un caso de uso

¿Cómo deberían descubrirse los casos de uso? Es típico no estar seguro si algo es un caso de uso válido (o de manera más práctica, útil). Las tareas se pueden agrupar a muchos niveles de granularidad, desde uno o unos pocos pasos pequeños, hasta actividades de nivel de empresa.

¿A qué nivel y alcance deberían expresarse los casos de uso?

Las siguientes secciones presentan las ideas sencillas de los procesos y objetivos del negocio elementales, como marco para la identificación de los casos de uso de una aplicación.

Casos de uso para los procesos del negocio elementales

¿Cuál de éstos es un caso de uso válido?

- Negociar un Contrato con el Proveedor
- Gestionar las Devoluciones
- Iniciar Sesión

Podría argumentarse que todos ellos son casos de uso a diferentes niveles, dependiendo de los límites del sistema, actores y objetivos. La evaluación de estos candidatos se presenta después de una introducción a los procesos del negocio elementales.

En lugar de preguntar en general: “¿qué es un caso de uso válido?”, una pregunta más relevante para el caso de estudio PDV es: ¿cuál es el nivel útil para expresar los casos de uso en el análisis de requisitos de la aplicación?

Guía: El caso de uso EBP

Para el análisis de requisitos de una aplicación informática, céntrese en los casos de uso al nivel de procesos del negocio elementales (EBPs, Elementary Business Processes).

EBP es un término que procede del campo de la ingeniería de procesos del negocio⁴, y se define como:

Una tarea realizada por una persona en un lugar, en un instante, como respuesta a un evento del negocio, que añade un valor cuantificable para el negocio y deja los datos en un estado consistente, ej. Autorizar Crédito, o Solicitar Precio (se perdió la fuente original).

Esto se puede tomar demasiado literalmente: ¿Está mal considerar un caso de uso como un EBP si se requieren dos personas, o si una persona tiene que pasear? Probablemente no; sin embargo, la impresión general de la definición es casi correcta. No se trata de un pequeño paso como “eliminar una línea de pedido” o “imprimir el documento”. Sino que el escenario principal de éxito está formado probablemente por cinco o diez pasos. No tarda días y múltiples sesiones, como “negociar un contrato con el proveedor”; es una tarea que se aborda en una única sesión. Probablemente dura entre unos pocos minutos y una hora. Como con la definición del UP, hace hincapié en añadir valor observable y cuantificable al negocio, y llega a un acuerdo en el que el sistema y los datos se encuentran en un estado estable y consistente.

⁴ EBP es parecido al término **tarea de usuario** en la ingeniería de usabilidad, aunque el significado es menos estricto en ese dominio.

Un error típico de los casos de uso es definir muchos casos de uso a un nivel muy bajo; es decir, como un paso simple, subfunción o subtarea en un EBP.

Violaciones razonables de la guía EBP

Aunque los casos de uso “base” de una aplicación deberían satisfacer la guía EBP, normalmente es útil crear “sub” casos de uso separados que representan subtarefas, o pasos, en un caso de uso base. Pueden existir casos de uso que no sean EBP; potencialmente existen muchos a un nivel inferior. La guía sólo se utiliza para encontrar el nivel dominante de casos de uso en el análisis de requisitos de una aplicación; esto es, el nivel en el que nos tenemos que centrar para nombrarlos y escribirlos.

Por ejemplo, una subtarea o extensión como “pago a crédito” podría repetirse en varios casos de uso base. Es conveniente separarlo en un caso de uso propio (que no satisface la guía EBP) y conectarlo a varios casos de uso base, para evitar duplicaciones del texto.

En el Capítulo 25 estudiaremos las cuestiones acerca de las relaciones de casos de uso.

Casos de uso y objetivos

Los actores tienen objetivos (o necesidades) y utilizan las aplicaciones para ayudarles a satisfacerlos. En consecuencia, un caso de uso de nivel EBP se denomina caso de uso de nivel de **objetivo de usuario**, para remarcar que sirve (o debería servir) para satisfacer un objetivo de un usuario del sistema, o el actor principal.

Esto nos lleva a recomendar el siguiente procedimiento:

1. Encontrar los objetivos de usuario.
2. Definir un caso de uso para cada uno.

Esto supone un ligero cambio de énfasis para el modelador de casos de uso. En lugar de preguntar: “¿Cuáles son los casos de uso?”, uno comienza preguntando: “¿Cuáles son tus objetivos?”. De hecho, el nombre del caso de uso para un objetivo de usuario debería reflejar dicho objetivo, para resaltar este punto de vista. Objetivo: capturar o procesar una venta; caso de uso: *Procesar Venta*.

Nótese que debido a esta simetría, la guía EBP se puede aplicar igualmente para decidir si un objetivo o un caso de uso se encuentran a un nivel adecuado.

Por tanto, he aquí una idea clave con respecto a la investigación de los objetivos de usuario frente a la investigación de casos de uso:

Imagine que estamos juntos en un taller de requisitos. Nos podríamos preguntar:

- “¿Qué haces?” (una pregunta bastante orientada al caso de uso) o;
- “¿Cuáles son tus objetivos?”

Es más probable que las respuestas a la primera pregunta reflejen soluciones y procedimientos actuales, y las complicaciones asociadas a ellos.

Las respuestas a la segunda pregunta, especialmente combinada con una investigación para ascender en la jerarquía de objetivos (“¿cuál es el objetivo de ese objetivo?”), abren la visión de soluciones nuevas y mejoradas, centradas en añadir valor al negocio, y llegar al corazón de lo que el personal involucrado quiere del sistema que se está estudiando.

Ejemplo: aplicación de la guía EBP

Un analista de sistemas responsable del descubrimiento de los requisitos del sistema NuevaEra, está investigando los objetivos de usuario. La conversación transcurre de esta forma durante un taller de requisitos:

Analista de sistemas: “¿Cuáles son algunos de sus objetivos en el contexto de uso de un sistema PDV?”

Cajero: “Uno, iniciar la sesión rápidamente. También, capturar las ventas.”

Analista de sistemas: “¿Cuál cree que es el objetivo de nivel más alto que motiva el inicio de sesión?”

Cajero: “Intento identificarme en el sistema, de este modo puede validar que estoy autorizado para utilizar el sistema que captura ventas y otras tareas.”

Analista de sistemas: “¿Más alto que ése?”

Cajero: “Evitar robos, alteración de datos, y mostrar información privada de la compañía.”

Obsérvese que la estrategia del analista de buscar de manera ascendente en la jerarquía de objetivos para encontrar los objetivos de usuario de un nivel superior que todavía satisfagan la guía EBP, para obtener el objetivo real detrás de la acción, y también para entender el contexto de los objetivos.

“Evitar robos...” es de un nivel superior a un objetivo de usuario; podría llamarse objetivo de empresa, y no es un EBP. Por tanto, aunque puede inspirar nuevas formas de pensar en el problema y las soluciones (como eliminar sistemas PDV y cajeros completamente), lo vamos a dejar a un lado por ahora.

Disminuyendo el nivel del objetivo a “identificarme en el sistema y ser validado” se acerca al nivel de objetivo de usuario. ¿Pero es el nivel EBP? No añade valor observable o cuantificable al negocio. Si el responsable del comercio pregunta: “¿Qué hiciste hoy?” y dices “¡Inicié la sesión 20 veces!”, no se impresionaría. En consecuencia, es un objetivo secundario, siempre disponible para hacer algo útil, y no es un EBP u objetivo de usuario. En cambio, “capturar una venta” cumple el criterio para ser un EBP u objetivo de usuario.

Otro ejemplo podría ser, en algunas tiendas existe un proceso denominado “abrir caja”, en el cual un cajero inserta su propia bandeja del cajón de caja en el terminal, inicia la sesión, e indica al sistema el dinero que hay en caja. *Abrir caja* es un caso de uso de nivel EBP (o nivel de objetivo de usuario); el paso de inicio de sesión, en lugar de ser un caso de uso de nivel EBP, es un objetivo de subfunción para llevar a cabo el objetivo de abrir caja.

Objetivos y casos de uso de subfunción

Aunque “identificarme y ser validado” (o “iniciar sesión”) se ha eliminado como objetivo de usuario, es un objetivo de nivel más bajo, denominado **objetivo de subfunción** —subobjetivos que dan soporte a un objetivo de usuario—. Sólo deberían escribirse casos de uso de manera ocasional para estos objetivos de subfunción, aunque es un problema típico que observan los expertos cuando se les pide que evalúen y mejoren (normalmente que simplifiquen) un conjunto de casos de uso.

No es ilegal escribir casos de uso para objetivos de subfunción, pero no siempre es útil, ya que añade complejidad al modelo de casos de uso; puede haber cientos de objetivos de subfunción —o casos de uso de subfunción— en un sistema.

Un punto importante es que el número y granularidad de los casos de uso influyen en el tiempo y la dificultad para entender, mantener y gestionar los requisitos.

El motivo válido, más común para representar un objetivo de subfunción como un caso de uso, es cuando la subfunción se repite o es una precondition en muchos casos de uso de nivel de objetivos de usuario. Este hecho se cumple probablemente en el caso de “identificarme y ser validado”, que es una precondition de la mayoría, si no todos, los otros casos de uso de nivel de objetivos de usuario.

En consecuencia, podría escribirse como el caso de uso *Autenticar Usuario*.

Objetivos y casos de uso pueden ser compuestos

Normalmente, los objetivos son compuestos, desde el nivel de empresa (“ser rentable”), que incluyen muchos objetivos intermedios a nivel de uso de la aplicación (“se capturan las ventas”), que a su vez incluyen objetivos de subfunción dentro de las aplicaciones (“la entrada es válida”).

De manera análoga, los casos de uso se pueden escribir a niveles diferentes para satisfacer estos objetivos, y pueden estar compuestos de casos de uso de nivel inferior.

Estos diferentes niveles de objetivos y casos de uso son una fuente típica de confusión en la identificación del nivel adecuado de los casos de uso de una aplicación. La guía EBP proporciona una orientación para eliminar casos de uso de nivel excesivamente bajo.

6.9. Descubrimiento de actores principales, objetivos y casos de uso

Los casos de uso se definen para satisfacer los objetivos de usuario de actores principales. Por tanto, el procedimiento básico es:

- 1. Elegir los límites del sistema. ¿Es sólo una aplicación software, el hardware y la aplicación como un todo, que lo utiliza más de una persona o una organización completa?

- 2. Identificar los actores principales —aquellos que tienen objetivos de usuario que se satisfacen mediante el uso de los servicios del sistema—.
- 3. Para cada uno, identificar sus objetivos de usuario. Elevarlos al nivel de objetivos de usuario más alto que satisfaga la guía EBP.
- 4. Definir los casos de uso que satisfagan los objetivos de usuario; nombrarlos de acuerdo con sus objetivos. Normalmente, los casos de uso del nivel de objetivo de usuario se corresponderán uno a uno con los objetivos de usuario, aunque hay al menos una excepción, como se verá.

Paso 1: Elegir el límite del sistema

Para este caso de estudio, el propio sistema PDV es el sistema que se está diseñando; todo lo que queda fuera de él, está fuera de los límites del sistema, incluyendo el cajero, el servicio de autorización de pagos, etcétera.

Si no está clara la definición de los límites del sistema que se está diseñando, se puede aclarar definiendo lo que está fuera —los actores principales externos y de apoyo—. Una vez identificados los actores externos, los límites se vuelven más claros. Por ejemplo, ¿se encuentra la responsabilidad de autorización de pagos completa en los límites del sistema? No, hay un actor del servicio externo de autorización de pagos.

Pasos 2 y 3: Identificar los actores principales y objetivos

Es artificial establecer de manera estricta que la identificación de los actores principales es antes que los objetivos de usuario; en un taller de requisitos, la gente pone en común sus ideas y dan lugar a una mezcla de ambos. Algunas veces los objetivos ponen de manifiesto a los actores, o viceversa.

Guía: Centrar la discusión en los actores principales en primer lugar, ya que establece el marco para las investigaciones posteriores.

Preguntas útiles para encontrar los actores principales y objetivos

Además de los actores principales y objetivos de usuario obvios, las preguntas siguientes ayudan a identificar otros que se podrían haber pasado:

- | | |
|---|--|
| ¿Quién arranca y para el sistema? | ¿Quién se encarga de la administración del sistema? |
| ¿Quién gestiona a los usuarios y la seguridad? | ¿Es un actor el “tiempo” porque el sistema hace algo como respuesta a un evento de tiempo? |
| ¿Existe un proceso de control que reinicie el sistema si falla? | ¿Quién evalúa la actividad o el rendimiento del sistema? |

- ¿Cómo se gestionan
las actualizaciones de software?
¿Actualizaciones automáticas o no?
- ¿Quién evalúa los registros?
¿Se recuperan de manera remota?

Actores principales y de apoyo

Recordemos que los actores principales tienen objetivos de usuario que se satisfacen mediante el uso de los servicios del sistema. Acuden al sistema para que les ayude. Al contrario que los *actores de apoyo*, que proporcionan servicios al sistema que se está diseñando. Por ahora, nos centraremos en encontrar los actores principales, no los de apoyo.

Recordemos también que los actores principales pueden ser —entre otras cosas— otros sistemas informáticos, como procesos software “guardianes” (“*watchdog*”).

Sugerencia

Desconfía si ninguno de los actores principales se corresponde con un sistema informático externo.

La lista actor-objetivo

Recoge los actores principales y sus objetivos de usuario en una lista actor-objetivo. En términos de los artefactos del UP, debería corresponderse con una sección del artefacto Visión (que se describirá en el capítulo siguiente).

Por ejemplo:

Actor	Objetivo	Actor	Objetivo
Cajero	procesar ventas procesar alquileres gestionar las devoluciones abrir caja cerrar caja ...	Administrador del Sistema	añadir usuarios modificar usuarios eliminar usuarios gestionar seguridad gestionar las tablas del sistema ...
Director	poner en marcha suspender operación ...	Sistema de Actividad de Ventas	analizar los datos de ventas y rendimiento
...	...	...	...

El Sistema de Actividad de Ventas es una aplicación remota a la que se le solicitará con frecuencia datos de ventas desde cada nodo PDV en la red.

Dimensión de la planificación del proyecto

En la práctica, esta lista tiene columnas adicionales para la prioridad, esfuerzo y riesgo; esto se tratará brevemente en el Capítulo 36.

La complicada realidad

Esta lista parece ordenada, pero la realidad de su creación es cualquier cosa salvo eso. Son necesarias muchas “tormentas de ideas” y discusiones durante los talleres de requisitos. Consideremos el ejemplo anterior que ilustraba la aplicación de la regla EBP al objetivo de “iniciar sesión”. Durante el taller, mientras se crea esta lista, el cajero podría proponer “iniciar sesión” como un objetivo de usuario. El analista de sistemas profundizará y subirá el nivel del objetivo, más allá del mecanismo de bajo nivel de inicio de sesión (el cajero posiblemente estaba pensando en utilizar un cuadro de diálogo en una GUI), al nivel de “identificar y autenticar al usuario”. Pero, el analista entonces se da cuenta de que no cumple la guía EBP, y lo descarta como objetivo de usuario. Por supuesto, la realidad es incluso algo diferente a esto puesto que un analista con experiencia cuenta con un conjunto de heurísticas, procedentes de experiencias o estudios anteriores, una de las cuales es “la autenticación de los usuarios es rara vez un EBP”, y de este modo es probable que se elimine rápidamente.

El actor principal y los objetivos de usuario dependen del límite del sistema

¿Por qué es el cajero, y no el cliente, el actor principal del caso de uso de *Procesar Venta*? ¿Por qué no aparece el cliente en la lista actor-objetivo?

La respuesta depende del límite del sistema que se esté diseñando, como se ilustra en la Figura 6.1. Si consideramos la empresa o el servicio de caja como un sistema agregado, el cliente *es* un actor principal, con el objetivo de obtener artículos o servicios, y marcharse. Sin embargo, desde el punto de vista de únicamente el sistema PDV (que es la elección del límite del sistema para este caso de estudio), éste atiende el objetivo del cajero (y de la tienda) de procesar la venta del cliente.

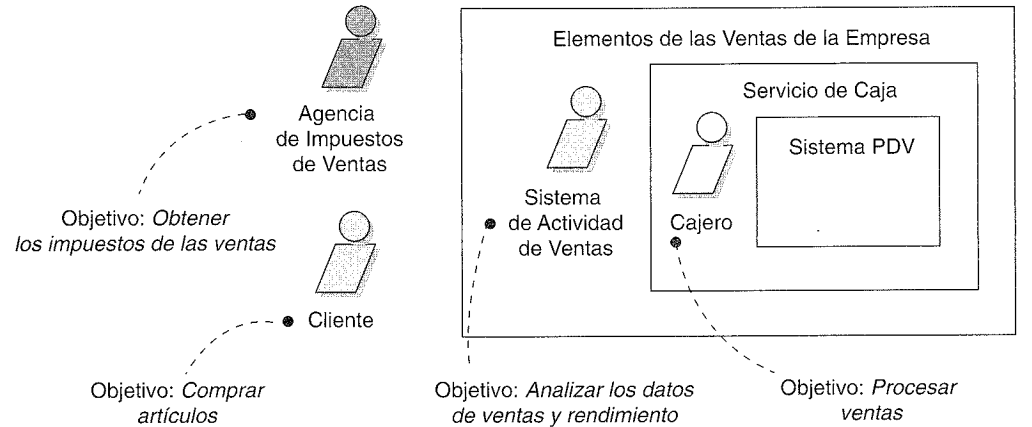


Figura 6.1. Actores principales y objetivos con diferentes límites del sistema.

Actores y objetivos por medio del análisis de eventos

Otro enfoque para ayudar en la búsqueda de los actores, objetivos y casos de uso, es identificar los eventos externos. ¿Cuáles son, de dónde proceden y por qué? A menudo,

un grupo de eventos pertenecen al mismo objetivo o caso de uso de nivel EBP. Por ejemplo:

Evento Externo	Parte del Actor	Objetivo
introducir línea de venta	Cajero	procesar una venta
introducir el pago	Cliente o Cajero	procesar una venta
...	...	

Paso 4: Definir los casos de uso

Por lo general, definimos un caso de uso de nivel EBP por cada objetivo de usuario. Nombramos el caso de uso de manera similar al objetivo de usuario; por ejemplo, Objetivo: procesar una venta; Caso de Uso: *Procesar Venta*.

También, nombre los casos de uso comenzando con un verbo.

Una excepción típica a un caso de uso por objetivo, es agrupar objetivos separados CRUD⁵ (crear, recuperar, actualizar, eliminar) en un caso de uso CRUD, llamado, por convención, *Gestionar<X>*. Por ejemplo, los objetivos “editar usuario”, “eliminar usuario”, etcétera, todos se satisfacen en el caso de uso *Gestionar Usuarios*.

“Definir casos de uso” comprende varios niveles de esfuerzo, variando desde unos pocos minutos, para recoger simplemente los nombres, hasta semanas, con el fin de escribir las versiones en formato completo. La última sección del proceso UP de este capítulo, sitúa este trabajo —cuándo y cuánto— en el contexto del desarrollo iterativo y del UP.

6.10. Enhorabuena: se han escrito los casos de uso y no son perfectos

La necesidad de comunicación y participación

El equipo del PDV NuevaEra va escribiendo casos de uso en múltiples talleres de requisitos, a través de una serie de iteraciones de desarrollo cortas, añadiendo incrementalmente al conjunto, y refinando y adaptando en base a la retroalimentación. Los expertos en la materia de estudio, los cajeros y programadores participan activamente en el proceso de escritura. No existen intermediarios entre los cajeros, otros usuarios y los desarrolladores, sino que se establece una comunicación directa.

Está bien, pero no es suficiente. Las especificaciones de requisitos escritas dan la impresión de ser correctas; pero no lo son. Los casos de uso y otros requisitos todavía

⁵ N. del T.: Acrónimo de Create-Retrieve-Update-Delte.

no serán correctos —garantizado—. Carecen de información crítica y contienen afirmaciones erróneas. La solución no es la aptitud del proceso “en cascada” de esforzarse mucho por recopilar los requisitos perfectos y completos al principio, aunque, por supuesto, lo hacemos lo mejor que podemos en el tiempo disponible. Pero nunca será suficiente.

Se necesita un enfoque diferente. Una buena parte del enfoque es el desarrollo iterativo, pero se necesita algo más: *comunicación personal continua*. Comunicación y participación cercana y continua —diaria— entre los desarrolladores y alguien que entienda el dominio y pueda tomar decisiones sobre los requisitos. Alguno de los programadores se puede acercar y en cuestión de segundos obtener aclaraciones, en cualquier momento que surja una duda. Por ejemplo, las prácticas de XP [Beck00] incluyen una excelente recomendación: *los usuarios deben tener dedicación a tiempo completo al proyecto, permaneciendo en la sala del proyecto*.

6.11. Escritura de casos de uso en un estilo esencial independiente de la interfaz de usuario

¡Nuevo y mejorado! Razones a favor de utilizar las huellas dactilares

Investigar y preguntar acerca de los objetivos, en lugar de las tareas y procedimientos fomenta que se centre la atención en la esencia de los requisitos —la intención detrás de ellos—. Por ejemplo, durante un taller de requisitos, el cajero podría decir que uno de sus objetivos es “iniciar la sesión”. El cajero, probablemente, estaría pensando en una GUI, cuadro de diálogo, identificador de usuario (ID) y contraseña (*password*). Éste es un mecanismo para alcanzar un objetivo, en lugar del objetivo en sí mismo. Mediante la investigación ascendente de la jerarquía de objetivos (“¿Cuál es el objetivo del objetivo?”), el analista del sistema llega hasta el objetivo independiente del mecanismo: “identificarse y conseguir la autenticación”, o incluso un objetivo de nivel superior: “prevenir robos...”.

Este proceso de descubrimiento puede abrir la visión a soluciones nuevas y mejoradas. Por ejemplo, los teclados y ratones con lectores biométricos, normalmente para las huellas dactilares son ahora habituales y baratos. Si el objetivo es “identificación y autenticación”, ¿por qué no hacerlo rápido y fácil, utilizando un lector biométrico en el teclado? Pero, la respuesta adecuada conlleva también un análisis de usabilidad, como conocer el perfil de los usuarios típicos del sistema. ¿Qué pasa si tienen los dedos cubiertos de grasa? ¿Tienen dedos?

Escritura en estilo esencial

Esta idea se ha resumido en varias guías de casos de uso como “no considere la interfaz de usuario; céntrese en la intención”. [Cockburn01]. La motivación y notación la ha estudiado Larry Constantine de manera más completa, en el contexto de la creación de interfaces de usuario (UIs) mejores y de la ingeniería de usabilidad [Constantine94,

CL99]. Constantine denomina al estilo de escritura **esencial** cuando evita los detalles de UI y se centra en la intención real del usuario⁶.

En un estilo de escritura esencial, la narración se expresa al nivel de las *intenciones* de los usuarios y *responsabilidades* del sistema, en lugar de sus acciones concretas. Son independientes de los detalles acerca de la tecnología y los mecanismos, especialmente aquellos relacionados con la UI.

Escriba casos de uso en un estilo esencial; no considere la interfaz de usuario y céntrese en la intención del actor.

Todos los ejemplos de casos de uso anteriores de este capítulo, como *Procesar Venta*, se escribieron con la intención de seguir un estilo esencial.

Nótese que el diccionario define *objetivo* como sinónimo de intención [MW89], lo que ilustra la conexión entre la idea del estilo *esencial* de Constantine y el punto de vista orientado al objetivo que se puso de relieve anteriormente en este capítulo. De hecho, muchos pasos de las *intenciones* de los actores, también se pueden caracterizar como *objetivos* de subfunción.

Ejemplos de contraste

Estilo esencial

Supongamos que el caso de uso *Gestionar Usuarios* requiere identificación y autenticación. El estilo esencial, inspirado en Constantine, utiliza el formato en dos columnas. Sin embargo, se puede escribir en una columna.

Intención del Actor	Responsabilidad del Sistema
1. El Administrador se identifica	2. Autenticar la identidad
3. ...	

En el formato de una columna esto se muestra como:

...
1. El Administrador se identifica
2. El Sistema autentica la identidad
3. ...

La solución de diseño para estas intenciones y responsabilidades está muy abierta: lectores biométricos, interfaces gráficas de usuario (GUIs), etcétera.

⁶ El término procede de los “modelos esenciales” en el *Análisis de Sistemas Esenciales* [MP84].

Estilo concreto—evitar durante el trabajo de requisitos inicial

En contraste, hay un estilo de **caso de uso concreto**. En este estilo, se incluyen en el texto del caso de uso las decisiones acerca de la interfaz de usuario. El texto podría incluso mostrar instantáneas de las pantallas de las ventanas, discutir la navegación de las ventanas, la manipulación de los elementos de la GUI, etcétera. Por ejemplo:

- ...
1. El Administrador introduce su ID y contraseña en el cuadro de diálogo (ver Dibujo 3).
2. El Sistema autentica al Administrador.
3. El Sistema muestra la ventana de “edición de usuarios” (ver Dibujo 4).
4. ...

Estos casos de uso concretos podrían ser útiles para ayudar en el diseño de la GUI detallada y concreta en etapas posteriores, pero no son adecuados durante el trabajo del análisis de requisitos inicial. Durante el trabajo de requisitos inicial, “no considere la interfaz de usuario, céntrese en la intención”.

6.12. Actores

Un actor es cualquier cosa con comportamiento, incluyendo el propio sistema que se está estudiando (SuD, *System under Discussion*) cuando solicita los servicios de otros sistemas⁷. Los actores principales y de apoyo aparecerán en los pasos de acción del texto del caso de uso. Los actores no son solamente roles que juegan personas, sino también organizaciones, software y máquinas. Hay tres tipos de actores externos con relación al SuD:

- **Actor principal:** tiene objetivos de usuario que se satisfacen mediante el uso de los servicios del SuD. Por ejemplo, el cajero.
 - ¿Por qué se identifica? Para encontrar los objetivos de usuario, los cuales dirigen los casos de uso.
- **Actor de apoyo:** proporciona un servicio (por ejemplo, información) al SuD. El servicio de autorización de pago es un ejemplo. Normalmente se trata de un sistema informático, pero podría ser una organización o una persona.
 - ¿Por qué se identifica? Para clarificar las interfaces externas y los protocolos.
- **Actor pasivo:** está interesado en el comportamiento del caso de uso, pero no es principal ni de apoyo; por ejemplo, la agencia tributaria del gobierno.
 - ¿Por qué se identifica? Para asegurar que *todos* los intereses necesarios se han identificado y satisfecho. Los intereses de los actores pasivos algunas veces son sutiles o es fácil no tenerlos en cuenta, a menos que estos actores sean identificados explícitamente.

⁷ Éste fue un refinamiento y mejora para las definiciones de actores alternativas, incluyendo las de las primeras versiones de UML y el UP [Cockburn97]. Las antiguas definiciones excluían, de manera inconsistente, el SuD como actor, incluso cuando recurría a servicios de otros sistemas. Todas las entidades podrían jugar múltiples *roles*, incluyendo el SuD.

6.13. Diagramas de casos de uso

UML proporciona notación para los diagramas de casos de uso con el fin de ilustrar los nombres de los casos de uso y los actores, y las relaciones entre ellos (ver Figura 6.2).

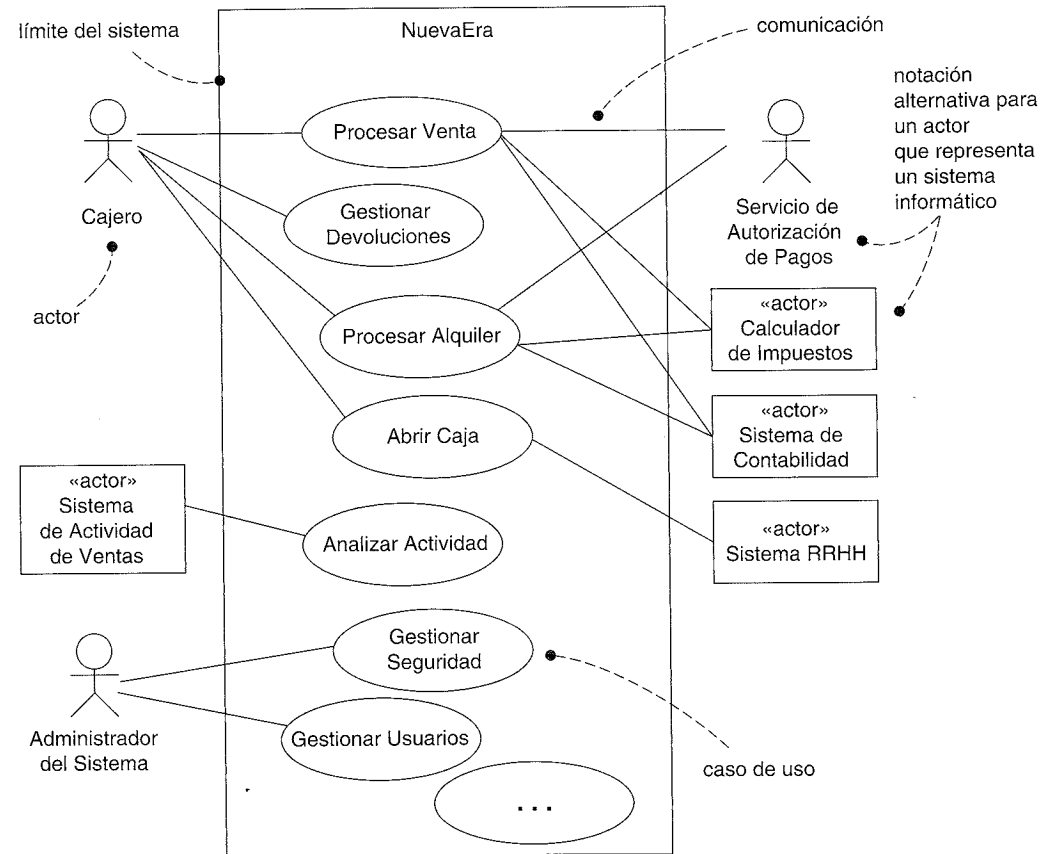


Figura 6.2. Diagrama de contexto de casos de uso parcial.

Los diagramas de caso de uso y las relaciones entre los casos de uso son secundarios en el trabajo con los casos de uso. Los casos de uso son documentos de texto. Trabajar con los casos de uso significa escribir texto.

Un signo típico de un modelador de casos de uso novato (o un estudiante) es la preocupación por los diagramas de casos de uso y las relaciones entre los casos de uso, en lugar de escribir texto. Los expertos mundiales en casos de uso, como Anderson, Fowler, Cockburn, entre otros, minimizan la importancia de los diagramas de casos de uso y las relaciones entre los casos de uso, y en lugar de eso se centran en escribir. Con eso como advertencia, un simple diagrama de casos de uso proporciona un conciso diagrama de contexto visual del sistema, que muestra los actores externos y cómo utilizan el sistema.

Sugerencia

Dibuje un diagrama de casos de uso sencillo junto con la lista actor-objetivo.

Un diagrama de casos de uso es una excelente representación del contexto del sistema; conforma un buen **diagrama de contexto**, esto es, muestra los límites de un sistema, lo que permanece fuera de él, y cómo se utiliza. Sirve como herramienta de comunicación que resume el comportamiento de un sistema y sus actores. La Figura 6.2 presenta una muestra de diagrama de contexto de casos de uso *parcial* para el sistema NuevaEra.

Sugerencias en la realización de los diagramas

La Figura 6.3 muestra algunos consejos sobre los diagramas. Nótese que la caja del actor contiene el símbolo «actor». Este símbolo se denomina **estereotipo UML**; se trata de un mecanismo para clasificar un elemento en cierto modo. El nombre de un estereotipo se escribe entre comillas francesas —signo ortográfico especial formado por un único carácter (no “<<” y “>>”) conocido sobre todo por su uso en la tipografía francesa para indicar una cita—.

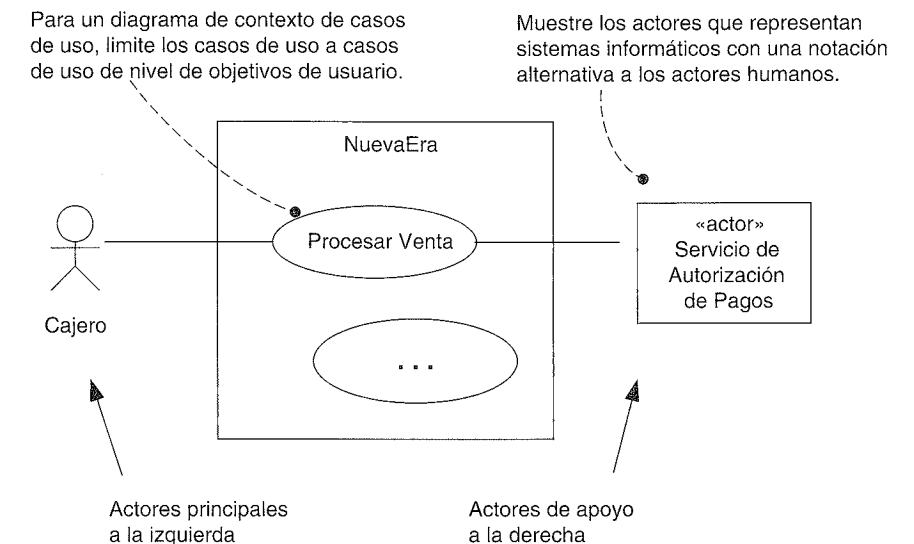


Figura 6.3. Sugerencias sobre la notación.

Para clarificar, algunos prefieren destacar los actores que se corresponden con sistemas informáticos externos con una notación alternativa, como se ilustra en la Figura 6.4.

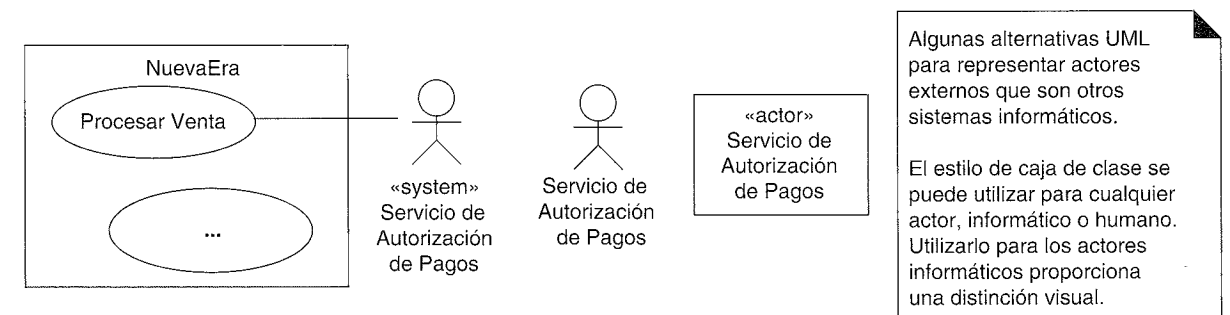


Figura 6.4. Notación alternativa para los actores.

Advertencia sobre el exceso de diagramas

Reiteramos que la importancia del trabajo de los casos de uso es escribir texto, no diagramas o centrarse en las relaciones entre los casos de uso. Si una organización dedica muchas horas (o peor, días) trabajando en los diagramas de casos de uso y debatiendo las relaciones en los casos de uso, en lugar de centrarse en escribir texto, se ha desaprovechado el esfuerzo.

6.14. Requisitos en contexto y listas de características de bajo nivel

Como queda implícito en el título del libro *Uses Cases: Requirements in Context* [GK00], una motivación clave de la idea de caso de uso es considerar y organizar los requisitos en el contexto de los objetivos y escenarios de uso de un sistema. Eso es bueno —mejora la cohesión y comprensión—. Sin embargo, los casos de uso no son los únicos artefactos de requisitos necesarios. Algunos requisitos no funcionales, reglas de dominio y contexto, y otros elementos difíciles de ubicar, se capturan mejor en la Especificación Complementaria, que se describirá en el siguiente capítulo.

Una idea detrás de los casos de uso es sustituir las listas de características de bajo nivel (típicas de los métodos de requisitos tradicionales) por los casos de uso (con algunas excepciones). Estas listas tendían a ser como sigue, normalmente agrupadas por áreas funcionales:

ID	Característica
CARAC1.9	El Sistema aceptará entradas de los identificadores de los artículos.
...	...
CARAC2.4	El Sistema registrará los pagos a crédito en el sistema de cuentas por cobrar.

De tales listas detalladas de las características de bajo nivel se puede aprovechar algo. Sin embargo, la lista completa no se recoge en media página; es más probable que ocupe docenas o cientos de páginas. Esto nos conduce a algunos obstáculos, que los casos de uso ayudan a abordar. Éstos incluyen:

- Listas de funciones largas y detalladas no relacionan los requisitos en un contexto cohesivo; las diferentes funciones y características aparecen progresivamente como una “lista de la lavandería” inconexa de elementos. En cambio, los casos de uso sitúan los requisitos en el contexto de las historias y objetivos de uso del sistema.
- Si se utilizan tanto los casos de uso como las listas de características detalladas, hay duplicación. Más trabajo, mayor volumen para escribir y leer, más problemas de consistencia y sincronización.

Sugerencia

Esfuércese en sustituir las listas detalladas de características de bajo nivel por casos de uso.

Son aceptables las listas de características de alto nivel del sistema

Es típico y útil resumir la funcionalidad del sistema con una lista breve de características de alto nivel, denominadas características del sistema, en un documento de Visión. A diferencia de las 100 páginas de características detalladas de bajo nivel, la lista de características del sistema tiende a incluir sólo unas pocas docenas de elementos. La lista proporciona un resumen muy conciso de la funcionalidad del sistema, independiente de la vista del caso de uso. Por ejemplo:

Resumen de características del sistema

- capturar las ventas.
- autorización del pago (crédito, débito, cheque).
- administración del sistema para los usuarios, seguridad, tablas de códigos y constantes, etcétera.
- procesamiento automático de ventas sin conexión, cuando fallan los componentes externos.
- transacciones en tiempo real, en base a los estándares industriales, con sistemas de terceras partes, que incluye inventario, contabilidad, recursos humanos, cálculo de impuestos y servicios de autorización de pagos.
- definición y ejecución de reglas de negocio adaptadas y que se pueden añadir en ejecución en puntos típicos, fijados en los escenarios de proceso.
- ...

Esto se estudiará en el siguiente capítulo.

¿Cuándo son apropiadas las listas de características detalladas?

Algunas veces los casos de uso no encajan realmente; algunas aplicaciones exigen un punto de vista dirigido por las características. Por ejemplo, los servidores de aplicaciones, productos de bases de datos y otros sistemas *middleware* o *back-end* necesitan ante todo ser considerados y evolucionar en términos de las *características* (“Necesitamos el soporte de XML en la siguiente versión”). Los casos de uso no se ajustan de manera natural a estas aplicaciones o al modo en que necesitan evolucionar en términos de las fuerzas del mercado.

6.15. Los casos de uso no son orientados a objetos

No hay nada orientado a objetos en los casos de uso; uno no está realizando un análisis orientado a objetos si escribe casos de uso. Esto no es un defecto sino una aclaración. De hecho, los casos de uso constituyen una herramienta para el análisis de requisitos ampliamente aplicable, que se puede utilizar en proyectos no orientados a objetos, lo cual

incrementa su utilidad como método de requisitos. Sin embargo, como estudiaremos, los casos de uso son una entrada fundamental en las actividades clásicas de A/DOO.

6.16. Casos de uso en el UP

Los casos de uso son vitales y centrales en el UP, que fomentan el **desarrollo dirigido por casos de uso**. Esto implica:

- Los requisitos se recogen principalmente en casos de uso (el Modelo de Casos de Uso); otras técnicas de requisitos (como las listas de funciones) son secundarias, si es que se utilizan.
- Los casos de uso son una parte importante de la planificación iterativa. El trabajo de una iteración se define —en parte— eligiendo algunos escenarios de caso de uso, o casos de uso completos. Los casos de uso son una entrada clave para hacer estimaciones.
- Las **realizaciones de los casos de uso** dirigen el diseño. Es decir, el equipo diseña objetos y subsistemas que colaboran para ejecutar o realizar los casos de uso.
- Los casos de uso, a menudo, influyen en la organización de los manuales de usuario.

El UP diferencia entre casos de uso del sistema y del negocio. Los **casos de uso del sistema** son los que se han estudiado en este capítulo, como *Procesar Venta*. Se crean en la disciplina Requisitos, y forman parte del Modelo de Casos de Uso.

Los **casos de uso del negocio** se escriben con menos frecuencia. Si se hace, se crean en la disciplina Modelado del Negocio, como parte de un esfuerzo de reingeniería de los procesos de negocio a gran escala, o para ayudar a entender el contexto de un nuevo sistema en el negocio. Describen una secuencia de acciones de un negocio como un todo para cumplir un objetivo de un **actor del negocio** (un actor en el entorno del negocio, como un cliente o un proveedor). Por ejemplo, en un restaurante, un caso de uso del negocio es *Servir una Comida*.

Casos de uso y especificación de requisitos a lo largo de las iteraciones

Esta sección reitera la idea clave del UP y el desarrollo iterativo: medir el tiempo y el nivel de esfuerzo de la especificación de requisitos a lo largo de las iteraciones. La Tabla 6.1 presenta una muestra (no una receta) de la estrategia del UP sobre el modo de desarrollar los requisitos.

Nótese que el equipo técnico comienza construyendo los fundamentos de la producción del sistema cuando, quizás, sólo se han detallado el 10% de los requisitos, y de hecho, se retrasa de manera deliberada la continuación del trabajo de los requisitos concertados hasta casi el final de la primera iteración de elaboración.

Ésta es la diferencia clave entre el proceso iterativo y el proceso en cascada: el desarrollo con calidad de producción de los fundamentos del sistema comienza rápidamente, mucho antes de conocer todos los requisitos.

Obsérvese que cerca del final de la primera iteración de elaboración, hay un segundo taller de requisitos, durante el que quizás, el 30% de los casos de uso se escriben en

Tabla 6.1. Muestra del esfuerzo de los requisitos a lo largo de las primeras iteraciones; no es una receta.

Disciplina	Artefacto	Comentarios y nivel de esfuerzo de los requisitos				
		Inicio 1 semana	Elab 1 4 semanas	Elab 2 4 semanas	Elab 3 3 semanas	Elab 4 3 semanas
Requisitos.	Modelo de Casos de Uso.	2 días de taller de requisitos. Se identifican por el nombre la mayoría de los casos de uso y se resumen en un párrafo breve. Sólo el 10% se escribe en detalle.	Cerca del final de esta iteración, tiene lugar un taller de requisitos de 2 días. Se obtiene un mejor entendimiento y retroalimentación a partir del trabajo de implementación, entonces se completa el 30% de los casos de uso en detalle.	Cerca del final de esta iteración, tiene lugar un taller de requisitos de dos días. Se obtiene una mejor comprensión y retroalimentación a partir del trabajo de implementación, entonces se completa el 50% de los casos de uso en detalle.	Repetir, se completa el 70% de todos los casos de uso en detalle.	Repetir con la intención de clarificar y escribir en detalle del 80-90% de los casos de uso. Sólo una pequeña parte de éstos se construyen durante la elaboración; el resto se aborda durante la construcción.
Diseño.	Modelo de Diseño.	Nada.	Diseño de un pequeño conjunto de requisitos de alto riesgo significativos desde el punto de vista de la arquitectura.	Repetir.	Repetir.	Repetir. Deberían ahora estabilizarse los aspectos de alto riesgo significativos para la arquitectura.
Implementación.	Modelo de Implementación (código, etc.)	Nada.	Implementar esto.	Repetir. Se construye el 5% del sistema final.	Repetir. Se construye el 10% del sistema final.	Repetir. Se construye el 15% del sistema final.
Gestión del Proyecto.	Plan de Desarrollo de SW.	Estimación muy imprecisa del esfuerzo total.	La estimación comienza a tomar forma.	Un poco mejor...	Un poco mejor...	Ahora se pueden establecer racionalmente la duración global del proyecto, los hitos más importantes, estimación del coste y esfuerzo.

detalle. Este análisis de requisitos escalonado se beneficia de la retroalimentación a partir de la construcción de un poco del núcleo del software. La retroalimentación incluye la evaluación del usuario, pruebas y “conocimiento de lo que no conocemos” mejorado. Es decir, el acto de construir software rápidamente hace que surjan suposiciones y preguntas que necesitan aclararse.

Momento de la creación de los artefactos del UP

La Tabla 6.2 muestra algunos de los artefactos del UP y un ejemplo de la planificación de sus momentos de comienzo y refinamiento. El Modelo de Casos de Uso comienza en la fase de inicio, con quizás sólo el 10% de los casos de uso escritos con algo de detalle. La mayoría se escriben incrementalmente a lo largo de las iteraciones de la fase de elaboración, de manera que, al final de la elaboración se ha escrito un gran cuerpo de casos de uso detallados y otros requisitos (en la Especificación Complementaria), proporcionando una base realista para hacer una estimación precisa hasta el final del proyecto.

Tabla 6.2. Muestra de los artefactos UP y evolución temporal. c - comenzar; r – refinar.

Disciplina	Artefacto Iteración →	Inicio I1	Elab. E1...En	Const. C1...Cn	Trans. T1...T2
Modelado del Negocio	Modelo del Dominio		c		
Requisitos	Modelo de Casos de Uso	c	r		
	Visión	c	r		
	Especificación Complementaria	c	r		
	Glosario	c	r		
Diseño	Modelo de Diseño		c	r	
	Documento de Arquitectura SW		c		
	Modelo de Datos		c	r	
Implementación	Modelo de Implementación		c	r	r
Gestión del Proyecto	Plan de Desarrollo SW	c	r	r	r
Pruebas	Modelo de Pruebas		c	r	
Entorno	Marco de Desarrollo	c	r		

Casos de uso en la fase de inicio

La siguiente discusión detalla la información presentada en la Tabla 6.1.

No todos los casos de uso se escriben en formato completo durante la fase de inicio. Más bien, suponga que se lleva a cabo un taller de requisitos durante dos días al comienzo del estudio de NuevaEra. La primera parte del día se dedica a identificar los objetivos y el personal involucrado, y especular sobre lo que queda dentro y fuera del alcance del proyecto. Se escribe una tabla de casos de uso actor-objetivo y se presenta con el proyector del ordenador. Se inicia el diagrama de contexto de casos de uso. Tras unas pocas horas, quizás se identifican unos 20 objetivos de usuario (y por tanto, casos de uso de nivel de usuario), que incluye *Procesar Venta*, *Gestionar Devoluciones*, etcétera. La mayoría de los casos de uso interesantes, complejos o arriesgados, se escriben en formato breve; cada uno escrito con una duración media de dos minutos. El equipo comienza a formarse un esquema de alto nivel de la funcionalidad del sistema.

Después de esto, entre el 10% y el 20% de los casos de uso que representan las funciones complejas principales, o que son especialmente arriesgadas en alguna dimensión, se escriben en formato completo; el equipo investiga con algo más de profundidad para entender mejor la magnitud, complejidad y demonios ocultos en el proyecto, a través de una pequeña muestra de casos de uso interesantes. Quizás esto puede implicar dos casos de uso: *Procesar Venta* y *Gestionar Devoluciones*.

Se utiliza una herramienta de gestión de requisitos integrada con un procesador de texto para la escritura, y el trabajo se muestra por medio de un proyector mientras el equipo colabora en el análisis y la escritura. Se escriben las listas de *Intereses* y *Personal Involucrado* para estos casos de uso, para descubrir requisitos más refinados (y quizás costosos) funcionales y no funcionales —o cualidades del sistema— claves, como la fiabilidad y el rendimiento.

El objetivo del análisis no es completar los casos de uso de manera exhaustiva, sino dedicar unas horas a comprenderlos mejor.

El promotor del proyecto necesita decidir si merece la pena un estudio profundo (esto es, la fase de elaboración). La intención del trabajo de inicio no es hacer este estudio, sino adquirir una idea de poca fidelidad (y claramente propensa a errores) acerca del alcance, riesgo, esfuerzo, viabilidad técnica, y análisis del negocio, para decir avanzar, dónde comenzar si se hace, o si parar.

Quizás la fase de inicio del proyecto NuevaEra duró cinco días. La combinación del taller de requisitos de dos días y su análisis de casos de uso breve, y otros estudios durante la semana, condujeron a tomar la decisión de continuar con la fase de elaboración para el sistema.

Casos de uso en la elaboración

La siguiente discusión detalla la información presentada en la Tabla 6.1.

Se trata de una fase de múltiples iteraciones de duración fija (por ejemplo, cuatro iteraciones) en las cuales se construyen incrementalmente partes del sistema arriesgadas, de alto valor o significativas desde el punto de vista de la arquitectura, y se identifican y clarifican la “mayoría” de los requisitos. La retroalimentación de los pasos concretos de programación influye e informa del conocimiento de los requisitos por parte del equipo, que se refina de manera iterativa y adaptable. Quizás se aborda un taller de requisitos de dos días en cada iteración —cuatro talleres—. Sin embargo, no se estudian todos los casos de uso en cada taller. Se priorizan; los primeros talleres se centran en un subconjunto de los casos de uso más importantes.

En cada siguiente taller de requisitos breve, es el momento de adaptar y refinar la visión de los requisitos principales, que serán inestables en las primeras iteraciones y se irán estabilizando en las últimas. Por tanto, hay una interacción iterativa entre el descubrimiento de los requisitos y la construcción de partes del software.

Durante cada taller de requisitos se refinan los objetivos de usuario y la lista de casos de uso. Se escriben, y reescriben, la mayoría de los casos de uso, en formato completo. Al final de la elaboración, se escriben en detalle del “80 al 90%” de los casos de uso. Para el sistema PDV con 20 casos de uso de nivel de objetivo de usuario, 15 o más de los más complejos y arriesgados deberían investigarse, escribirse y reescribirse en formato completo.

Nótese que la elaboración conlleva programar partes del sistema. Al final de esta etapa, el equipo NuevaEra no sólo debería tener una mejor definición de los casos de uso, sino también algo de software ejecutable de calidad.

Casos de uso en la construcción

La etapa de construcción está compuesta de iteraciones de duración fija (por ejemplo, 20 iteraciones de dos semanas cada una) que se centra en completar el sistema, una vez que las principales cuestiones arriesgadas e inestables se han establecido en la elaboración. Tendrá lugar todavía la escritura de casos de uso menores y quizás talleres de requisitos,

pero mucho menos de lo que se hizo en la elaboración. En esta etapa, la mayoría de los requisitos funcionales y no funcionales principales deberían haberse estabilizado de manera iterativa y adaptable. La intención no es dar a entender que los requisitos se congelan o el estudio termina, sino que el grado de cambio es mucho menor.

6.17. Caso de estudio: casos de uso en la fase de inicio de NuevaEra

Como se ha descrito en la sección anterior, no todos los casos de uso se escriben en el formato completo durante la fase de inicio. El Modelo de Casos de Uso de esta fase para el caso de estudio podría detallarse como sigue:

Completo	Informal	Breve
Procesar Venta	Procesar Alquiler	Abrir Caja
Gestionar Devoluciones	Analizar Actividad de Ventas	Cerrar Caja
	Gestionar Seguridad	Gestionar Usuarios
	...	Poner en Marcha
		Suspender Operación
		Gestionar Tablas del Sistema
		...

6.18. Lecturas adicionales

La guía de casos de uso de mayor éxito, traducida a varios idiomas es *Writing Effective Use Cases* [Cockburn01]⁸. Por buenas razones se ha convertido en el libro de casos de uso más ampliamente leído y seguido y, por tanto, se recomienda como referencia fundamental. Este capítulo se basa, y es consistente, con su contenido. Sugerencia: no rechace el libro como consecuencia del uso de iconos para los diferentes niveles de casos de uso por parte de los autores, o el énfasis temprano en los niveles y la taxonomía de casos de uso. Los iconos son opcionales y no muy importantes. Y aunque el debate acerca de los niveles y objetivos podría parecer al principio que distrae la atención a aquellos nuevos en los casos de uso, los que han trabajado con ellos durante algún tiempo estiman que el nivel y alcance de los casos de uso son cuestiones prácticas claves, porque su incomprensión es una fuente típica de complicaciones en el modelado de casos de uso.

“Structuring Use Cases with Goals” [Cockburn97] es el artículo sobre casos de uso más citado, disponible on-line en www.usecases.org.

Use Cases: Requirements in Context [GK00] es otro texto útil. Destaca el importante punto de vista —como establece el título— de que los casos de uso no son únicamente

⁸ Nótese que Cockburn rima con *slow burn* (N. del T.: En una comunicación personal, el autor nos ha comentado que introdujo esta aclaración a petición de A. Cockburn para señalar que su apellido no se pronuncia como cock-burn y evitar chistes por la connotación sexual.)

otro artefacto de los requisitos, sino que constituyen el vehículo central que dirige el trabajo de los requisitos y la información.

Otra lectura que merece la pena destacar es *Applying Use Cases: A Practical Guide* [SW98], escrita por un profesor y experto en casos de uso que entiende y comunica cómo aplicar los casos de uso en un ciclo de vida iterativo.

6.19. Artefactos UP y contexto del proceso

Como se ilustra en la Figura 6.5 los casos de uso influyen en muchos artefactos UP.

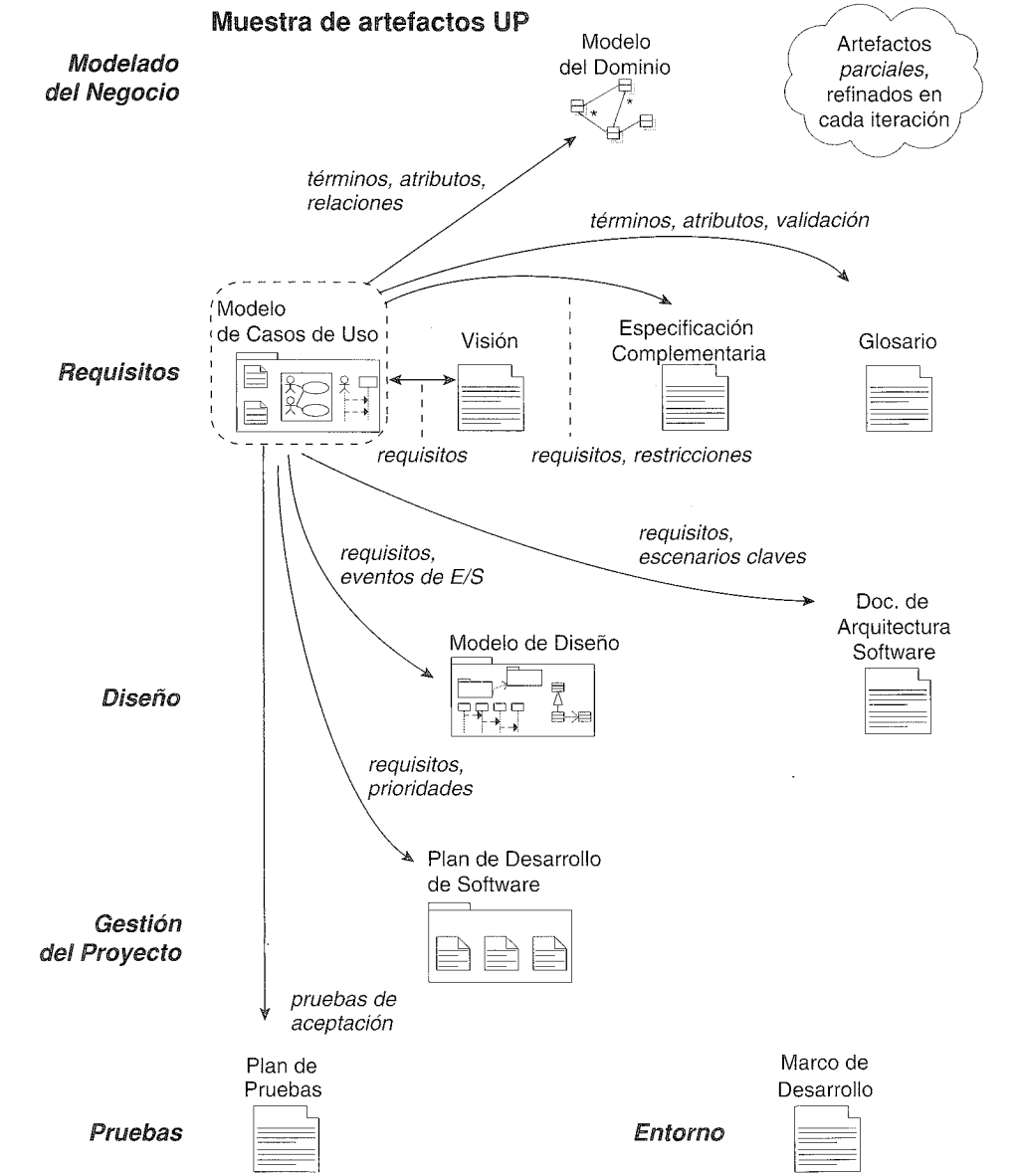


Figura 6.5. Muestra de la influencia entre los artefactos UP.

En el UP, el trabajo de los casos de uso es una actividad de la disciplina de requisitos que podría inicializarse durante un taller de requisitos. La Figura 6.6 presenta algunos consejos acerca del momento y el lugar para llevar a cabo este trabajo.

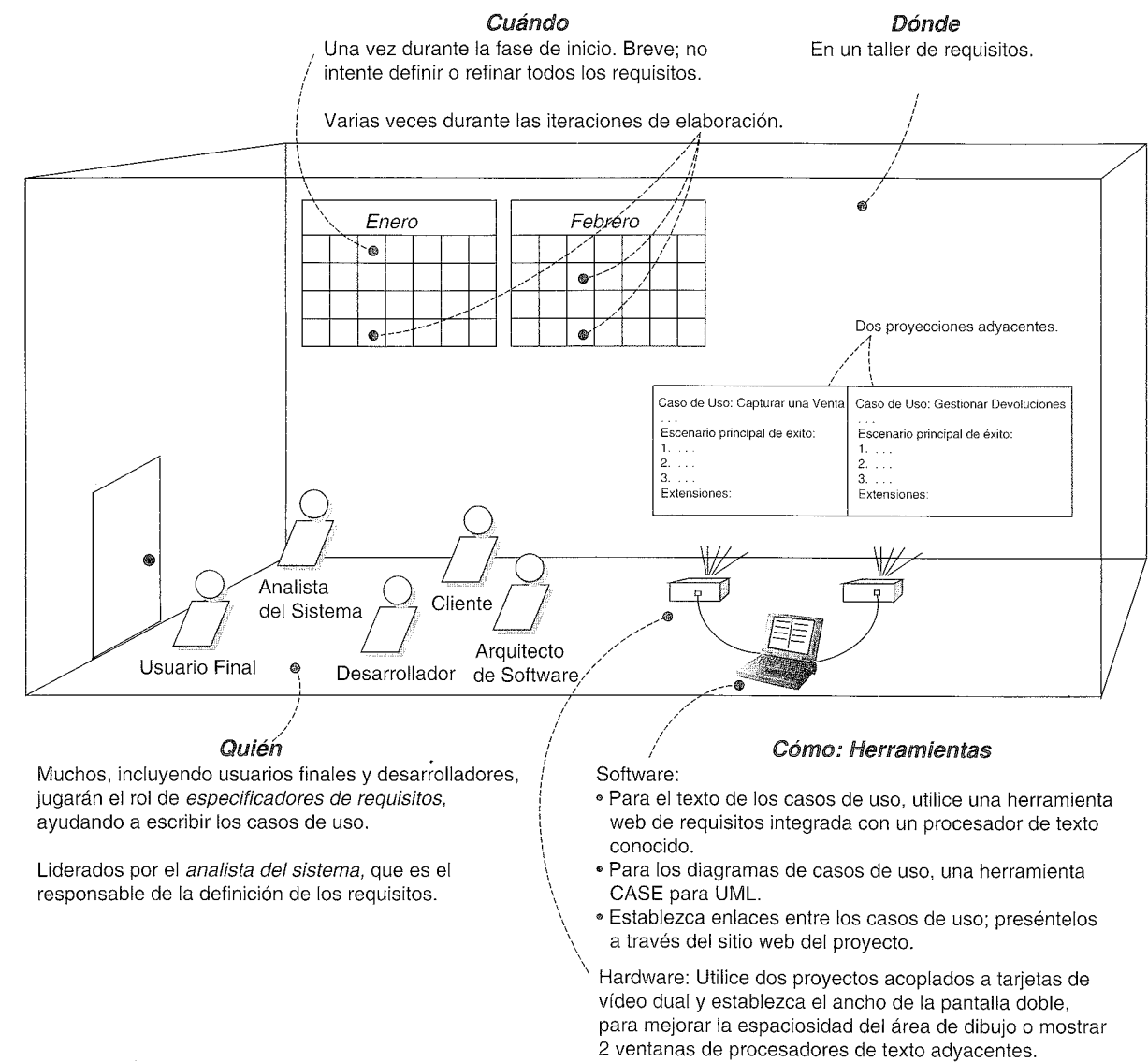


Figura 6.6. Proceso y establecimiento del contexto.

IDENTIFICACIÓN DE OTROS REQUISITOS

Cuando las ideas fallan, las palabras vienen muy bien.
Johann Wolfgang von Goethe

Objetivos

- Escribir una Especificación Complementaria, Glosario y Visión.
- Comparar y contrastar las características del sistema con los casos de uso.
- Relacionar la Visión con otros artefactos y con el desarrollo iterativo.
- Definir los atributos de calidad.

Introducción

No es suficiente escribir casos de uso. Existen otros tipos de requisitos que son necesarios identificar, como los relacionados con documentación, empaquetado, soporte, licencia, etcétera. Éstos se recogen en la **Especificación Complementaria**.

El **Glosario** almacena los términos y definiciones; puede también jugar el rol de diccionario de datos.

La **Visión** resume la “visión” del proyecto. Sirve para comunicar de manera concisa las grandes ideas acerca de por qué se propuso el proyecto, cuáles son los problemas, quiénes son las personas involucradas, qué necesitan, y cuál podría ser la apariencia de la solución propuesta.

Citando literalmente:

La Visión define la vista que tienen las personas involucradas del producto que se va a desarrollar, especificada en términos de las necesidades y características claves de dichas personas. Al contener un esquema de los principales requisitos previstos, proporciona la base contractual para los requisitos técnicos más detallados [RUP].

7.1. Ejemplos del PDV NuevaEra

El objetivo de los siguientes ejemplos no es presentar de manera exhaustiva la Visión, el Glosario y Especificación Complementaria, ya que algunas de las secciones —aunque útiles para el proyecto— no son relevantes para los objetivos de aprendizaje¹. La finalidad del libro son las principales técnicas en el diseño de objetos, análisis de requisitos con casos de uso, análisis orientado a objetos, no los problemas del PDV o las sentencias de Visión. Por tanto, sólo se hace referencia brevemente a algunas secciones, para establecer las conexiones entre el trabajo previo y futuro, destacar las cuestiones que merecen la pena, proporcionar una idea del contenido, y avanzar rápidamente.

7.2. Ejemplo NuevaEra: Especificación Complementaria (Parcial)

Especificación Complementaria			
Historia de revisiones			
Versión	Fecha	Descripción	Autor
Borrador de Inicio	10 Enero, 2031	Primer borrador. Para refinarse principalmente durante la elaboración.	Craig Larman

Introducción

Este documento es el repositorio de todos los requisitos del PDV NuevaEra que no se capturan en los casos de uso.

Funcionalidad

(Funcionalidad común entre muchos casos de uso)

Registro y gestión de errores

Registrar todos los errores en almacenamiento persistente.

¹ El crecimiento gradual del alcance no es sólo un problema de los requisitos, sino de la *escritura* sobre requisitos.

Reglas de negocio conectables

En varios puntos de los escenarios de varios casos de uso (pendientes de ser definidos) soportar la capacidad de adaptar la funcionalidad del sistema con un conjunto arbitrario de reglas que se ejecutan en ese punto o evento.

Seguridad

Todo uso requiere la autenticación de los usuarios.

Facilidad de uso

Factores humanos

El cliente será capaz de ver la información en un gran monitor del PDV. Por tanto:

- Se debe ver el texto fácilmente a una distancia de 1 metro.
- Evitar colores asociados con formas comunes de daltonismo.

Velocidad, comodidad y procesamiento libre de errores, son lo más importante en el procesamiento de ventas, ya que el comprador desea irse rápidamente, o perciben la experiencia de compra (y al vendedor) como menos positiva.

El cajero está mirando a menudo al cliente o los artículos, no a la pantalla del ordenador. Por tanto, se deben comunicar las señales y avisos con sonidos, en lugar de sólo mediante gráficos.

Fiabilidad

Capacidad de recuperación

Si se produce algún fallo al usar un servicio externo (autorización de pago, sistema de contabilidad,...) intentar solucionarlo con una solución local (ej. almacenar y remitir) para, no obstante, completar un venta. Se necesita mucho más análisis aquí...

Rendimiento

Como se mencionó en los factores humanos, los compradores quieren completar el proceso de ventas *muy* rápido. Un cuello de botella potencial es la autorización de pagos externa. El objetivo es conseguir la autorización en menos de 1 minuto, el 90% de las veces.

Soporte

Adaptabilidad

Los diferentes clientes del PDV NuevaEra tienen necesidades de procesamiento y reglas de negocio únicas en el procesamiento de una vena. Por tanto, en varios puntos definidos en el escenario (por ejemplo, cuando se inicia una nueva venta, cuando se añade una nueva línea de venta) se habilitarán reglas de negocio conectables.

Facilidad para cambiar la configuración

Clientes diferentes desean que varíe la configuración de la red para sus sistemas PDV, como clientes gruesos *versus* delgados, en dos capas *versus* N-capas, etcétera. Además, desean poder cambiar estas configuraciones, para reflejar sus cambios en el negocio y necesidades de rendimiento. Por tanto, el sistema será configurable para reflejar estas necesidades. Se necesita mucho más análisis en este área para descubrir las áreas y grado de flexibilidad, y el esfuerzo para conseguirla.

Restricciones de Implementación

La dirección de NuevaEra insiste en una solución utilizando las tecnologías Java, previendo que esto mejorará a largo plazo la portabilidad y soporte, además de facilitar el desarrollo.

Componentes adquiridos

- El sistema de cálculo de impuestos. Debe soportar sistemas de cálculo conectables de diferentes países.

Componentes de libre distribución

En general, recomendamos maximizar el uso de componentes Java de libre distribución en este proyecto.

Aunque es prematuro diseñar y elegir los componentes de manera definitiva, sugerimos los siguientes como candidatos posibles:

- Framework para registros JLog
- ...

Interfaces

Interfaces y hardware destacables

- Monitor con pantalla táctil (detectado por el sistema operativo como monitor corriente, y los movimientos de contacto como eventos del ratón)
- Escáner láser de código de barras (normalmente unido a un teclado especial, y el software considera las entradas escaneadas como entradas del teclado)
- Impresora de recibos
- Lector de tarjetas de crédito/débito
- Lector de firmas (pero no en la primera versión)

Interfaces software

Para la mayoría de los sistema de colaboración externos (calculador de impuestos, contabilidad, inventario, ...) necesitamos ser capaces de conectar diversos sistemas y, por tanto, diversas interfaces.

Reglas de dominio (negocio)

ID	Regla	Grado de variación	Fuente
REGLA1	Se requiere la firma para pagos a crédito. años la mayoría de los clientes	Se continuará solicitando la "firma" de los compradores, aunque en 2 años los clientes solicitarán la firma en un aparato de captura digital, y en 5 años esperamos que se demande la nueva "firma" digital única soportada por la ley americana.	La política de prácticamente todas las compañías de autorización de crédito.
REGLA2	Reglas sobre los impuestos. Hay que añadir un impuesto	Alto. La ley sobre impuestos cambia anualmente, en todos los niveles de gobierno.	ley

continúa

ID	Regla	Grado de variación	Fuente
	a las ventas. Ver los estatutos del gobierno para los detalles actuales.		
REGLA3	Las devoluciones de los pagos a crédito sólo pueden efectuarse como crédito en las cuentas de crédito de los compradores, no en efectivo.	Bajo.	Política de la compañía sobre la autorización de crédito.
REGLA4	Reglas de descuento al comprador. Ejemplo: Empleado: 20% Clientes preferentes: 10% Antiguos: 15%	Alto. Cada tienda utiliza reglas diferentes.	Política de la tienda.
REGLA5	Reglas de descuento de venta (nivel de transacción) Se aplica al total antes de los impuestos. Por ejemplo: 10% de descuento si el total es mayor de 100 €. 5% de descuento los lunes. 10% de descuento en las ventas de hoy entre las 10am y las 3pm. 50% de descuento en Tofu hoy desde las 9am hasta las 10am.	Alto. Cada tienda utiliza reglas distintas, y pueden cambiar diariamente o cada hora.	Política de la tienda.
REGLA6	Reglas de descuento de artículo (nivel de línea de venta). Ejemplo: 10% de descuento en tractores esta semana. Comprando 2 hamburguesas vegetales llévase 1 gratis.	Alto. Cada tienda utiliza reglas distintas, y pueden cambiar diariamente o cada hora.	Política de la tienda.

Cuestiones legales

Recomendamos algunos componentes de libre distribución si sus restricciones de licencia se pueden resolver para permitir la reventa de artículos que incluyen software de libre distribución. Todas las reglas de impuestos se tienen que aplicar, por ley, durante las ventas. Nótese que pueden cambiar con frecuencia.

Información en dominios de interés

Fijación de precios

Además de las reglas de fijación de precios que se describen en la sección de reglas del dominio, observe que los artículos tienen un *precio original*, y opcionalmente, un *precio rebajado*. El precio de los artículos (antes de los descuentos adicionales) es el precio rebajado, si existe. Las organizaciones mantienen el precio original incluso si hay un precio rebajado, por razones de contabilidad e impuestos.

Gestión de pagos a crédito y débito

Cuando el servicio de autorización de pagos aprueba un pago electrónico, de crédito o débito, son los responsables del pago al vendedor, no el comprador. En consecuencia, por cada pago, el vendedor necesita registrar la suma de dinero que le deben en las cuentas, desde el servicio de autorización. Normalmente, por las noches, el servicio de autorización llevará a cabo una transferencia electrónica de fondos a la cuenta de los vendedores por la suma de dinero total del día, menos un (pequeño) cargo por transacción que cobra el servicio.

Impuesto de ventas

Los cálculos de los impuestos de las ventas pueden ser muy complejos, y pueden cambiar regularmente como respuesta a la legislación en todos los niveles de gobierno. Por tanto, es aconsejable delegar el cálculo a software de terceras partes (de los que hay varios disponibles). Los impuestos se pueden deber a la ciudad, al gobierno regional y agencias nacionales. Algunos artículos podrían estar exentos de impuestos sin requisitos, o exentos dependiendo del comprador o el receptor objetivo (por ejemplo, un granjero o un niño).

Identificadores de artículos: UPCs, EANs, SKUs, código de barras y lectores de códigos de barras

El PDV NuevaEra necesita dar soporte a varios esquemas de identificador de artículos. UPCs (Códigos de Producto Universales, *Universal Product Codes*), EANs (Numeración de Artículos Europea, *European Article Numbering*) y SKUs (Unidades de Mantenimiento de Stock, *Stock Keeping Units*) son tres tipos comunes de sistemas de identificación para los artículos que se venden. El Número de Artículo Japonés (JANs, *Japanese Article Number*) es un tipo de la versión EAN.

SKUs son identificadores completamente arbitrarios definidos por el vendedor.

Sin embargo, los UPCs y EANs tienen un componente estándar y normativo. Diríjase a www.adams1.com/pub/russadam/upccode.html para obtener una buena visión general. También puede ver www.uc-council.org y www.ean-int.org.

7.3. Comentario: Especificación Complementaria

La Especificación Complementaria captura otros requisitos, información y restricciones que no se recogen fácilmente en los casos de uso o el Glosario, que comprende los atributos o requisitos de calidad “URPS+” de todo el sistema. Fíjese que los requisitos específicos de un caso de uso pueden (y probablemente deben) escribirse en primer lugar con el caso de uso, en una sección *Requisitos Especiales*, pero algunos prefieren también reunirlos en la Especificación Complementaria. Los elementos de la Especificación Complementaria podrían comprender:

- Requisitos FURPS+ —funcionalidad, facilidad de uso, fiabilidad, rendimiento y soporte—.
- Informes.
- Restricciones de software y hardware (sistemas operativos y de red...).
- Restricciones de desarrollo (por ejemplo, herramientas de proceso y desarrollo).
- Otras restricciones de diseño e implementación.

- Cuestiones de internacionalización (unidades, idiomas...).
- Documentación (usuario, instalación, administración) y ayuda.
- Licencia y otras cuestiones legales.
- Empaquetado.
- Estándares (técnico, seguridad, calidad).
- Cuestiones del entorno físico (por ejemplo, calor o vibración).
- Cuestiones operacionales (por ejemplo, ¿cómo se gestionan los errores o con qué frecuencia se hacen las copias de seguridad?).
- Reglas del dominio o negocio.
- Información en los dominios de interés (por ejemplo, ¿cuál es el ciclo completo de gestión de pagos a crédito?).

Las **restricciones** no son comportamientos, sino otro tipo de restricciones del diseño o el proyecto. También son requisitos, pero se denominan comúnmente “restricciones” para remarcar su influencia *restrictiva*. Por ejemplo:

Debe utilizar Oracle (tenemos un contrato de licencia con ellos)

Debe funcionar bajo Linux (disminuirá el coste)

Sugerencia

Las decisiones y restricciones de diseño tempranas (“elaboración prematura”) son casi siempre una mala idea, así que merece la pena desconfiar y cuestionarlas, especialmente durante la fase de inicio, cuando se ha analizado muy poco con cuidado. Algunas restricciones se imponen por causas inevitables, como restricciones legales, o una interfaz de un sistema externo existente al que se debe invocar.

Atributos de calidad

Algunos requisitos se denominan **atributos de calidad** [BCK98] (o “-ilities”) de un sistema. Éstos incluyen facilidad de uso (*usability*), fiabilidad (*reliability*), etcétera. Nótese que se refieren a cualidades del sistema, no a que estos requisitos sean necesariamente de calidad alta (la palabra está sobrecargada en inglés). Por ejemplo, la cualidad de soporte (*supportability*) podría elegirse deliberadamente que fuese baja si el producto no se pretende que sirva a largo plazo.

Hay de dos tipos:

1. Observables en la ejecución (funcionalidad, facilidad de uso, fiabilidad, rendimiento, etc.).
2. No observables en ejecución (soporte, pruebas, etc.).

La funcionalidad se especifica en los casos de uso, como otros atributos de calidad relacionados con casos de uso específicos (por ejemplo, las cualidades de rendimiento en el caso de uso *Procesar Venta*).

Otros atributos de calidad FURPS+ del sistema se describen en la Especificación Complementaria.

Aunque la funcionalidad es un atributo de calidad válido, en su uso común, el término “atributo de calidad” casi siempre se refiere a “cualidades del sistema que no sean la funcionalidad”. En este libro, el término se utiliza de este modo. Esto no es exactamente lo mismo que los requisitos no funcionales, que es un término más amplio que incluye *todo* excepto la funcionalidad (por ejemplo, empaquetado y licencia).

Cuando nos ponemos nuestro “gorro de arquitecto”, los atributos de calidad de todo el sistema (y por tanto, la Especificación Complementaria donde se recogen) son especialmente interesantes porque —como veremos en el Capítulo 32— el análisis y el diseño de la arquitectura tienen que ver en gran medida con la identificación y resolución de los atributos de calidad en el contexto de los requisitos funcionales. Por ejemplo, suponga que uno de los atributos de calidad es que el sistema NuevaEra debe ser bastante tolerante a fallos cuando fallen los servicios remotos. Desde el punto de vista de la arquitectura, esto tendrá una influencia global en las decisiones de diseño a gran escala.

Entre los atributos de calidad existen interdependencias, lo que implica compromisos. Un ejemplo sencillo en el PDV podría ser, “muy fiable (tolerante a fallos)” y “fácil de probar” son opuestos, puesto que hay muchas formas sutiles de que pueda fallar un sistema distribuido.

Reglas del dominio (negocio)

Las reglas del dominio [Ross97, GK00] dictan el modo en el que podría operar un dominio o negocio. No son requisitos de ninguna aplicación, aunque, a menudo, los requisitos de una aplicación se ven afectados por las reglas del dominio. Las políticas de la compañía, leyes físicas y leyes gubernamentales, son reglas de dominio típicas.

Se denominan comúnmente **reglas del negocio**, que son el tipo más común, pero ese término está limitado, ya que existen aplicaciones software que no son de gestión de un negocio, como simulación del clima o logística militar. Una simulación del clima incluye “reglas del dominio”, relacionadas con las leyes y relaciones físicas, que afectan a los requisitos de la aplicación.

Con frecuencia, resulta útil identificar y registrar aquellas reglas del dominio que afectan a los requisitos, normalmente materializados en los casos de uso, porque pueden clarificar el contenido de un caso de uso ambiguo o incompleto. Por ejemplo, en el PDV NuevaEra, si alguien pregunta si en el caso de uso *Procesar Venta* debería escribirse una alternativa a los pagos a crédito que no necesite la captura de la firma, existe una regla del negocio (REGLA1) que aclara que no se permitirá por ninguna compañía de autorización de crédito.

Advertencia

Las reglas no son requisitos de la aplicación. No registre las características del sistema como reglas. Las reglas describen las restricciones y comportamientos del modo de trabajar del dominio, no de la aplicación.

Información en los dominios de interés

A menudo, a los expertos del dominio que se está estudiando, les resulta útil escribir (o proporcionar URLs con) explicaciones de dominios relacionados con el nuevo sistema software (ventas y contabilidad, la geofísica de los flujos subterráneos de petróleo/agua/gas,...), para presentar el contexto y ayudar a la comprensión por parte del equipo de desarrollo. Podría contener referencias importantes a expertos o a literatura, fórmulas, leyes u otras referencias. Por ejemplo, los misterios de los esquemas de codificación UPC y EAN, y la interpretación del código de barras, los deben entender, en cierta medida, el equipo de NuevaEra.

7.4. Ejemplo NuevaEra: Visión (Parcial)

Visión

Historia de revisiones

Versión	Fecha	Descripción	Autor
Borrador de Inicio	10 Enero, 2031	Primer borrador. Para refinarse principalmente durante la elaboración.	Craig Larman

El análisis del ejemplo es ilustrativo, pero ficticio

Introducción

Prevemos una aplicación de punto de venta (PDV) tolerante a fallos de próxima generación, PDV NuevaEra, con flexibilidad para poder soportar variación en las reglas del negocio del cliente, múltiples mecanismos de terminal e interfaz de usuario, y la integración con múltiples sistemas de terceras partes.

Orientación

Oportunidad del negocio

Los productos PDV existentes no son adaptables al negocio del cliente, en términos de permitir variar las reglas de negocio y los diseños de la red (por ejemplo cliente delgado o no, arquitectura en 2, 3 ó 4 capas). Además, no permiten su extensión de manera adecuada cuando se incrementan los terminales y crece el negocio. Y ninguno permite trabajar en línea o desconectados, adaptándose dinámicamente dependiendo de los fallos. Ninguno se integra fácilmente con muchos sistemas de terceras partes. Ninguno admite nuevas tecnologías como los PDAs móviles. El mercado se siente insatisfecho debido a este estado inflexible de cosas, y demandan un PDV que rectifique esta situación.

Enunciado del problema

Los sistemas PDV tradicionales son inflexibles, intolerantes a fallos, y difíciles de integrar con sistemas de terceras partes. Esto da lugar a problemas en el oportuno procesamiento de las ventas, en el establecimiento de procesos mejorados que no concuerdan con el software, y con los datos de contabilidad e inventario precisos y oportunos para dar soporte a la planificación y medidas, entre otras cuestiones. Esto afecta a los cajeros, encargados del almacén, administradores del sistema y a la gestión empresarial.

Enunciado de la posición en el mercado del producto

— Resumen conciso de a quién está dirigido el producto, las características relevantes, y qué lo diferencia de la competencia.

Alternativas y competencia...

Descripción del personal involucrado

Demografía de mercado...

Resumen del personal involucrado (No usuarios)...

Resumen de Usuarios...

Objetivos de alto nivel y problemas claves del personal involucrado

El taller de requisitos de un día con los expertos en la materia que se está estudiando y otras personas involucradas, y encuestas a varios distribuidores, nos llevaron a los siguientes objetivos y problemas claves:

Objetivo de alto nivel	Prioridad	Problemas e inquietudes	Soluciones actuales
Procesamiento de ventas rápido, robusto e integrado.	Alta	Se reduce la velocidad cuando se incrementa la carga. Pérdida de la capacidad de procesamiento de las ventas si los componentes fallan. Carencia de información actualizada y precisa de la contabilidad y otros sistemas debido a que no está integrado con los sistemas de contabilidad, inventario y RRHH. Da lugar a dificultades en las medidas y la planificación. Imposibilidad de adaptar las reglas de negocio a requisitos del negocio únicos. Dificultad al añadir nuevos tipos de terminales o interfaces de usuario (por ejemplo, PDAs móviles).	Los productos PDV existentes permiten el procesamiento básico de ventas, pero no abordan estos problemas.
...	...	...	...

Objetivos de nivel de usuario

Los usuarios (y los sistemas externos) necesitan un sistema para satisfacer sus objetivos:

- *Cajero*: procesar las ventas, gestionar las devoluciones, abrir y cerrar caja.
- *Administrador del sistema*: gestionar los usuarios, gestionar la seguridad, gestionar las tablas del sistema.
- *Director*: poner en marcha, suspender operación.
- *Sistema de actividad de ventas*: analizar los datos de las ventas.
- ...

Entender quiénes son los participantes y sus problemas

Reunir las entradas de la Lista Actor-Objetivos y la sección de los casos de uso de Intereses del personal involucrado

Podría ser la Lista Actor-Objetivo creada durante el modelado de casos de uso o un resumen más conciso

Entorno de usuario...

Visión general del producto

Perspectiva del producto

El PDV NuevaEra residirá, normalmente, en tiendas; si se utilizan terminales móviles se encontrarán muy próximos a la red de la tienda, en el interior o en el exterior. Proporcionará servicios al usuario, y colaborará con otros sistemas, como se indica en la Figura Visión-1.

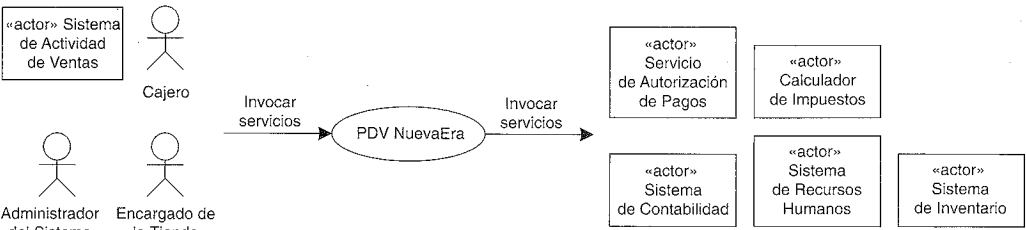


Figura Visión-1. Diagrama de contexto del sistema PDV NuevaEra.

Resumido a partir del diagrama de casos de uso

Los diagramas aparecen con diversos detalles, pero todos muestran los actores externos importantes para el sistema

Similar a la lista Actor-Objetivo, esta tabla relaciona objetivos, beneficios y soluciones, pero a un nivel más alto, no únicamente relacionado con los casos de uso

Resume el valor y características diferenciadoras del producto

Como se presenta abajo, las características del sistema constituyen un formato conciso para resumir la funcionalidad

Resumen de los beneficios

Característica soportada	Beneficio del personal involucrado
Funcionalmente, el sistema proporcionará todos los servicios típicos que requiere la organización de las ventas, incluyendo la entrada de las ventas, autorización de pagos, gestión de devoluciones, etcétera.	Servicios de punto de venta rápidos y automáticos.
Detección de fallos automática, cambiando a procesamiento local sin conexión cuando los servicios no estén disponibles.	Procesamiento de ventas continuado cuando fallan los componentes externos.
Reglas de negocio "conectables" en varios puntos del escenario durante el procesamiento de las ventas.	Configuración flexible de la lógica del negocio.
Transacciones en tiempo real con sistemas de terceras partes, haciendo uso de protocolos estándares industriales.	Información de ventas, contabilidad e inventario oportuna y precisa, para abordar las mediciones y la planificación.
...	...

Suposiciones y dependencias...

Coste y fijación del precio...

Licencia e instalación...

Resumen de las características del sistema

- Entrada de ventas.
- Autorización de pagos (crédito, débito, cheque).